```
# Install dependencies and mount Drive
!pip install pyreadstat pandas numpy -q

import google.colab.drive as drive
drive.mount('/content/drive')
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 41.6 MB/s eta 0:00:00
Mounted at /content/drive
```

```
import pandas as pd

file_path = "/content/drive/MyDrive/pmuy_data.csv"

df = pd.read_csv(file_path, low_memory=False)

df.shape
```

```
(1238208, 19)
```

```
!pip install pyarrow
```

```
Requirement already satisfied: pyarrow in /usr/local/lib/python3.12/dist-packages (18.1.0)
```

```
import pandas as pd

# Original file
file_path = "/content/drive/MyDrive/pmuy_data.csv"

# Read CSV
df = pd.read_csv(file_path, low_memory=False)

# Check shape
print(df.shape)

# ----------------------------------------------------
# OPTION 1: Compress CSV using gzip (no data loss)
# ----------------------------------------------------

compressed_csv_path = "/content/drive/MyDrive/pmuy_data_compressed.csv.gz"

df.to_csv(
    compressed_csv_path,
    index=False,
    compression="gzip"
)

print("Compressed CSV saved successfully!")
```

```
(1238208, 19)
Compressed CSV saved successfully!
```

```
import pandas as pd

file_path = "/content/drive/MyDrive/pmuy_data_compressed.csv.gz"
df = pd.read_csv(file_path)

df.head()
```

	hhid	hv005	hv009	hv024	hv025	hv201	hv204	hv206	hv213	hv219	hv220	hv226	hv270	sh34	sh36	hv106_
0	1000101	191072	4	andaman and nicobar islands	urban	piped into dwelling	on premises	yes	cement	male	51	lpg, natural gas	middle	christian	none of above	educati presch
1	1000109	191072	3	andaman and nicobar islands	urban	piped to yard/plot	on premises	yes	cement	female	40	lpg, natural gas	richer	hindu	NaN	prim
2	1000110	191072	4	andaman and nicobar islands	urban	piped to yard/plot	on premises	yes	cement	male	38	lpg, natural gas	richer	hindu	none of above	second
3	1000111	191072	3	andaman and nicobar islands	urban	piped into dwelling	on premises	yes	cement	male	46	lpg, natural gas	richer	hindu	none of above	second
4	1000117	191072	2	andaman and nicobar islands	urban	piped into dwelling	on premises	yes	cement	male	28	kerosene	middle	hindu	NaN	prim

```
import pandas as pd
import numpy as np

df = pd.read_csv('/content/drive/MyDrive/pmuy_data_compressed.csv.gz')

print("Shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
print("\nDtypes:")
print(df.dtypes)
```

```
Shape: (1238208, 19)

Column names:
['hhid', 'hv005', 'hv009', 'hv024', 'hv025', 'hv201', 'hv204', 'hv206', 'hv213', 'hv219', 'hv220', 'hv226', 'hv270', 'sh34', 'sh

Dtypes:
hhid         int64
hv005        int64
hv009        int64
hv024       object
hv025       object
hv201       object
hv204       object
hv206       object
hv213       object
hv219       object
hv220       object
hv226       object
hv270       object
sh34        object
sh36        object
hv106_01    object
state       object
survey       int64
post         int64
dtype: object
```

```
print("Shape:", df.shape)
print("\nColumn names:")
print("First few rows:", df.head())
```

```
Shape: (1238208, 19)

Column names:
First few rows:       hhid   hv005  hv009                        hv024  hv025  \
0  1000101  191072      4  andaman and nicobar islands  urban
1  1000109  191072      3  andaman and nicobar islands  urban
2  1000110  191072      4  andaman and nicobar islands  urban
3  1000111  191072      3  andaman and nicobar islands  urban
4  1000117  191072      2  andaman and nicobar islands  urban
```

```
                hv201        hv204 hv206   hv213   hv219 hv220  \
0  piped into dwelling  on premises   yes  cement    male    51
1   piped to yard/plot  on premises   yes  cement  female    40
2   piped to yard/plot  on premises   yes  cement    male    38
3  piped into dwelling  on premises   yes  cement    male    46
4  piped into dwelling  on premises   yes  cement    male    28

             hv226   hv270       sh34           sh36  \
0  lpg, natural gas  middle  christian  none of above
1  lpg, natural gas  richer      hindu            NaN
2  lpg, natural gas  richer      hindu  none of above
3  lpg, natural gas  richer      hindu  none of above
4         kerosene  middle      hindu            NaN

                 hv106_01                         state  survey  post
0  no education, preschool  andaman and nicobar islands       4     0
1                  primary  andaman and nicobar islands       4     0
2                secondary  andaman and nicobar islands       4     0
3                secondary  andaman and nicobar islands       4     0
4                  primary  andaman and nicobar islands       4     0
```

```
missing = pd.DataFrame({
    'missing_count': df.isna().sum(),
    'missing_pct':   df.isna().mean() * 100
}).sort_values('missing_pct', ascending=False)

print(missing[missing['missing_count'] > 0])
```

```
      missing_count  missing_pct
sh36          51867     4.188876
```

```
print("=== sh36 (caste) value counts ===")
print(df['sh36'].value_counts(dropna=False))

print("\n=== sh34 (religion) value counts ===")
print(df['sh34'].value_counts(dropna=False))

print("\n=== Missing caste by round ===")
print(df.groupby('survey')['sh36'].apply(lambda x: x.isna().sum()))

print("\n=== Missing caste by state ===")
print(df.groupby('state')['sh36'].apply(lambda x: x.isna().mean()*100).sort_values(ascending=False).head(10))
```

```
=== sh36 (caste) value counts ===
sh36
other backward class    459929
none of above           249636
scheduled tribe         237543
scheduled caste         231436
NaN                      51867
don't know                7797
Name: count, dtype: int64

=== sh34 (religion) value counts ===
sh34
hindu                    930484
muslim                   145651
christian                 98840
sikh                      26939
buddhist/neo-buddhist     17602
other                     15686
jain                       1911
no religion                 897
parsi/zoroastrian           173
jewish                       25
Name: count, dtype: int64

=== Missing caste by round ===
survey
4    24227
5    27640
Name: sh36, dtype: int64

=== Missing caste by state ===
state
jammu and kashmir              29.192021
goa                            27.729384
assam                          23.550612
west bengal                    20.194546
tripura                        14.822084
andaman and nicobar islands     9.231686
```

```
karnataka                      6.180578
manipur                        5.309870
arunachal pradesh              4.725559
meghalaya                      4.074392
Name: sh36, dtype: float64
```

```
# Using inplace parameter
df['sh36'].fillna('99', inplace=True)

print("Caste distribution after filling:")
print(df['sh36'].value_counts())
print(f"\nMissing remaining: {df['sh36'].isna().sum()}")
```

```
Caste distribution after filling:
sh36
other backward class    459929
none of above           249636
scheduled tribe         237543
scheduled caste         231436
99                       51867
don't know                7797
Name: count, dtype: int64

Missing remaining: 0
/tmp/ipykernel_7758/2469074944.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are sett

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df


  df['sh36'].fillna('99', inplace=True)
```

```
df.isna().sum()
```

	0
hhid	0
hv005	0
hv009	0
hv024	0
hv025	0
hv201	0
hv204	0
hv206	0
hv213	0
hv219	0
hv220	0
hv226	0
hv270	0
sh34	0
sh36	0
hv106_01	0
state	0
survey	0
post	0

dtype: int64

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/content/drive/MyDrive/pmuy_data_compressed.csv.gz')

# ============================================================
```

```
# WEIGHTS: hv005 (DHS standard: divide by 1,000,000)
# ============================================================
df['weight'] = df['hv005'] / 1_000_000

print("=== WEIGHTS ===")
print(f"Weight range: {df['weight'].min():.4f} to {df['weight'].max():.4f}")
print(f"Weight mean: {df['weight'].mean():.4f}")
print(f"Sum of weights: {df['weight'].sum():.0f} (estimated population in household units)")
print(f"Sample size (N): {len(df):,}")
print(f"Design effect: {df['weight'].sum() / len(df):.2f}")  # How much weights inflate sample

# ============================================================
# CLEAN FUEL: hv226 → binary (CORRECT ORDER)
# ============================================================

CLEAN_FUELS = ['electricity', 'lpg, natural gas', 'biogas']

# Step 1: Drop households that don't cook food
df = df[~df['hv226'].isin(['no food cooked in house'])]

# Step 2: Create clean fuel binary (only for cooking households)
df['clean_fuel'] = df['hv226'].isin(CLEAN_FUELS).astype(int)

# Step 3: Drop any remaining NaN (if any exist)
df = df[df['clean_fuel'].notna()]

print("\n=== CLEAN FUEL ===")
print(df['clean_fuel'].value_counts())
print(f"\nClean fuel rate (unweighted): {df['clean_fuel'].mean()*100:.2f}%")

# Weighted clean fuel rate
weighted_clean = np.average(df['clean_fuel'], weights=df['weight']) * 100
print(f"Clean fuel rate (weighted): {weighted_clean:.2f}%")
```

```
=== WEIGHTS ===
Weight range: 0.0007 to 38.9548
Weight mean: 1.0000
Sum of weights: 1238208 (estimated population in household units)
Sample size (N): 1,238,208
Design effect: 1.00

=== CLEAN FUEL ===
clean_fuel
0    671767
1    564185
Name: count, dtype: int64

Clean fuel rate (unweighted): 45.65%
Clean fuel rate (weighted): 51.53%
```

```
# ============================================================
# CHECK STATES ACROSS ROUNDS
# ============================================================
print("\n" + "="*60)
print("STATE CONSISTENCY CHECK")
print("="*60)

states_nfhs4 = set(df[df['post']==0]['state'].unique())
states_nfhs5 = set(df[df['post']==1]['state'].unique())

print(f"States in NFHS-4: {len(states_nfhs4)}")
print(f"States in NFHS-5: {len(states_nfhs5)}")

states_in_both = states_nfhs4.intersection(states_nfhs5)
print(f"\nStates in both: {len(states_in_both)}")

states_only_4 = states_nfhs4 - states_nfhs5
if states_only_4:
    print(f"\n⚠ States only in NFHS-4: {sorted(states_only_4)}")

states_only_5 = states_nfhs5 - states_nfhs4
if states_only_5:
    print(f"\n⚠ States only in NFHS-5: {sorted(states_only_5)}")

if not states_only_4 and not states_only_5:
    print("\n✓ Perfect! All states in both rounds")
```

```
============================================================
STATE CONSISTENCY CHECK
============================================================
States in NFHS-4: 35
States in NFHS-5: 35

States in both: 35

✓ Perfect! All states in both rounds
```

```
import pandas as pd
import numpy as np

print("="*60)
print("CREATING BINARY CONTROL VARIABLES")
print("="*60)

# ============================================================
# 1. RURAL (binary) - from hv025
# ============================================================
if 'hv025' in df.columns:
    df['rural'] = df['hv025'].str.lower().str.strip().map({
        'rural': 1,
        'urban': 0
    }).fillna(0).astype(int)
    print("✓ Created 'rural' (1=rural, 0=urban)")

# ============================================================
# 2. ELECTRICITY (binary) - from hv206
# ============================================================
if 'hv206' in df.columns:
    df['electricity'] = df['hv206'].str.lower().str.strip().map({
        'yes': 1,
        'no': 0
    }).fillna(0).astype(int)
    print("✓ Created 'electricity' (1=has electricity, 0=no)")

# ============================================================
# 3. FEMALE HEAD (binary) - from hv219
# ============================================================
if 'hv219' in df.columns:
    df['female_head'] = df['hv219'].str.lower().str.strip().map({
        'female': 1,
        'male': 0
    }).fillna(0).astype(int)
    print("✓ Created 'female_head' (1=female head, 0=male head)")

# ============================================================
# 4. IMPROVED WATER (binary) - from hv201
# ============================================================
if 'hv201' in df.columns:
    improved_water_sources = [
        'piped into dwelling', 'piped to yard/plot', 'public tap/standpipe',
        'tube well or borehole', 'protected well', 'protected spring',
        'rainwater', 'bottled water'
    ]
    df['improved_water'] = df['hv201'].str.lower().str.strip().isin(improved_water_sources).astype(int)
    print("✓ Created 'improved_water' (1=improved source, 0=unimproved)")

# ============================================================
# 5. IMPROVED FLOOR (binary) - from hv213
# ============================================================
if 'hv213' in df.columns:
    unimproved_floors = [
        'mud/clay/earth', 'dung', 'sand', 'raw wood planks',
        'palm, bamboo', 'stone'
    ]
    df['improved_floor'] = (~df['hv213'].str.lower().str.strip().isin(unimproved_floors)).astype(int)
    print("✓ Created 'improved_floor' (1=improved, 0=unimproved)")

# ============================================================
# 6. CASTE DUMBIRS - from sh36
# ============================================================
if 'sh36' in df.columns:
    # Fill missing as 'not stated'
    df['caste'] = df['sh36'].fillna('not stated').str.lower().str.strip()
```

```
    # Create dummies
    caste_dummies = pd.get_dummies(df['caste'], prefix='caste', drop_first=False)

    # Select relevant caste categories
    caste_categories = ['scheduled caste', 'scheduled tribe', 'other backward class', 'none of above']
    for category in caste_categories:
        col_name = f'caste_{category.replace(" ", "_")}'
        if col_name in caste_dummies.columns:
            df[col_name] = caste_dummies[col_name].astype(int)
        else:
            df[f'caste_{category.replace(" ", "_")}'] = 0

    print("✓ Created caste dummies: 'caste_scheduled_caste', 'caste_scheduled_tribe', 'caste_other_backward_class', 'caste_non

# ============================================================
# 7. RELIGION DUMBIRS - from sh34
# ============================================================
if 'sh34' in df.columns:
    df['religion'] = df['sh34'].str.lower().str.strip()

    # Create dummies for major religions
    religion_dummies = pd.get_dummies(df['religion'], prefix='religion')

    major_religions = ['hindu', 'muslim', 'christian', 'sikh', 'buddhist']
    for religion in major_religions:
        col_name = f'religion_{religion}'
        if col_name in religion_dummies.columns:
            df[col_name] = religion_dummies[col_name].astype(int)
        else:
            df[col_name] = 0

    print("✓ Created religion dummies: 'religion_hindu', 'religion_muslim', 'religion_christian', 'religion_sikh', 'religion_bu

# ============================================================
# 8. EDUCATION OF HEAD (binary for higher education) - from hv106_01
# ============================================================
if 'hv106_01' in df.columns:
    df['head_edu'] = df['hv106_01'].str.lower().str.strip()
    # Higher education = secondary or above
    df['head_higher_edu'] = df['head_edu'].isin(['secondary', 'higher']).astype(int)
    # At least primary education
    df['head_primary_edu'] = df['head_edu'].isin(['primary', 'secondary', 'higher']).astype(int)
    print("✓ Created 'head_higher_edu' (1=secondary+), 'head_primary_edu' (1=primary+)")

# ============================================================
# 9. PIPED WATER (specific binary) - from hv201
# ============================================================
if 'hv201' in df.columns:
    piped_sources = ['piped into dwelling', 'piped to yard/plot']
    df['piped_water'] = df['hv201'].str.lower().str.strip().isin(piped_sources).astype(int)
    print("✓ Created 'piped_water' (1=piped to dwelling/yard, 0=other)")

# ============================================================
# 10. WEALTH QUINTILE (keep as numeric, but create rich dummy)
# ============================================================
if 'hv270' in df.columns:
    # Convert wealth to numeric if needed
    wealth_map = {
        'poorest': 1, 'poorer': 2, 'middle': 3, 'richer': 4, 'richest': 5
    }
    if df['hv270'].dtype == 'object':
        df['wealth_quintile'] = df['hv270'].str.lower().str.strip().map(wealth_map)
    else:
        df['wealth_quintile'] = pd.to_numeric(df['hv270'], errors='coerce')

    # Top 2 quintiles as rich
    df['rich'] = (df['wealth_quintile'] >= 4).astype(int)
    print("✓ Created 'wealth_quintile' (1-5) and 'rich' (1=quintile 4-5)")

# ============================================================
# VERIFY CREATED VARIABLES
# ============================================================
print("\n" + "="*60)
print("CREATED BINARY VARIABLES SUMMARY")
print("="*60)
```

```
binary_vars = ['rural', 'electricity', 'female_head', 'improved_water',
               'improved_floor', 'head_higher_edu', 'head_primary_edu',
               'piped_water', 'rich']

for var in binary_vars:
    if var in df.columns:
        print(f"\n{var}:")
        print(f"  Mean: {df[var].mean()*100:.1f}%")
        print(f"  Value counts:\n{df[var].value_counts().to_dict()}")

# ============================================================
# LIST ALL CONTROLS FOR REGRESSION
# ============================================================
print("\n" + "="*60)
print("CONTROL VARIABLES FOR DiD REGRESSION")
print("="*60)

control_vars = []
for var in ['rural', 'electricity', 'female_head', 'improved_water',
            'improved_floor', 'head_higher_edu', 'piped_water', 'rich',
            'wealth_quintile', 'caste_scheduled_caste', 'caste_scheduled_tribe',
            'caste_other_backward_class', 'religion_muslim', 'religion_christian']:
    if var in df.columns:
        control_vars.append(var)

print(f"Available controls: {len(control_vars)}")
for i, var in enumerate(control_vars, 1):
    print(f"  {i}. {var}")

# ============================================================
# OPTIONAL: SAVE DATAFRAME WITH NEW VARIABLES
# ============================================================
df.to_csv('data_with_binary_controls.csv', index=False)
print("\n✓ Saved: data_with_binary_controls.csv")
```

```
============================================================
CREATING BINARY CONTROL VARIABLES
============================================================
✓ Created 'rural' (1=rural, 0=urban)
✓ Created 'electricity' (1=has electricity, 0=no)
✓ Created 'female_head' (1=female head, 0=male head)
✓ Created 'improved_water' (1=improved source, 0=unimproved)
✓ Created 'improved_floor' (1=improved, 0=unimproved)
✓ Created caste dummies: 'caste_scheduled_caste', 'caste_scheduled_tribe', 'caste_other_backward_class', 'caste_none_of_above'
✓ Created religion dummies: 'religion_hindu', 'religion_muslim', 'religion_christian', 'religion_sikh', 'religion_buddhist'
✓ Created 'head_higher_edu' (1=secondary+), 'head_primary_edu' (1=primary+)
✓ Created 'piped_water' (1=piped to dwelling/yard, 0=other)
✓ Created 'wealth_quintile' (1-5) and 'rich' (1=quintile 4-5)

============================================================
CREATED BINARY VARIABLES SUMMARY
============================================================

rural:
  Mean: 72.9%
  Value counts:
{1: 900949, 0: 335003}

electricity:
  Mean: 92.4%
  Value counts:
{1: 1141833, 0: 94119}

female_head:
  Mean: 15.9%
  Value counts:
{0: 1039530, 1: 196422}

improved_water:
  Mean: 83.0%
  Value counts:
{1: 1026303, 0: 209649}

improved_floor:
  Mean: 57.1%
  Value counts:
{1: 706069, 0: 529883}

head_higher_edu:
  Mean: 50.9%
  Value counts:
```

```
{1: 628645, 0: 607307}

head_primary_edu:
  Mean: 69.4%
  Value counts:
{1: 857999, 0: 377953}

piped_water:
  Mean: 32.4%
  Value counts:
{0: 835876, 1: 400076}
```

```
 # ============================================================
# CHECK UNIQUE STATES (Debug the 36 vs 35 issue)
# ============================================================

print("="*60)
print("DEBUG: CHECKING UNIQUE STATES")
print("="*60)

unique_states = df['state'].unique()
print(f"Total unique states in dataset: {len(unique_states)}")
print(f"States: {sorted(unique_states)}")

# Check if any state has both '37' and 'odisha' or similar issues
for state in unique_states:
    print(f"  '{state}' - Type: {type(state)}")
```

```
============================================================
DEBUG: CHECKING UNIQUE STATES
============================================================
Total unique states in dataset: 35
States: ['andaman and nicobar islands', 'andhra pradesh', 'arunachal pradesh', 'assam', 'bihar', 'chandigarh', 'chhattisgarh', '
  'andaman and nicobar islands' - Type: <class 'str'>
  'andhra pradesh' - Type: <class 'str'>
  'arunachal pradesh' - Type: <class 'str'>
  'assam' - Type: <class 'str'>
  'bihar' - Type: <class 'str'>
  'chandigarh' - Type: <class 'str'>
  'chhattisgarh' - Type: <class 'str'>
  'dadra and nagar haveli and daman and diu' - Type: <class 'str'>
  'goa' - Type: <class 'str'>
  'gujarat' - Type: <class 'str'>
  'haryana' - Type: <class 'str'>
  'himachal pradesh' - Type: <class 'str'>
  'jammu and kashmir' - Type: <class 'str'>
  'jharkhand' - Type: <class 'str'>
  'karnataka' - Type: <class 'str'>
  'kerala' - Type: <class 'str'>
  'lakshadweep' - Type: <class 'str'>
  'madhya pradesh' - Type: <class 'str'>
  'maharashtra' - Type: <class 'str'>
  'manipur' - Type: <class 'str'>
  'meghalaya' - Type: <class 'str'>
  'mizoram' - Type: <class 'str'>
  'nagaland' - Type: <class 'str'>
  'delhi' - Type: <class 'str'>
  'odisha' - Type: <class 'str'>
  'puducherry' - Type: <class 'str'>
  'punjab' - Type: <class 'str'>
  'rajasthan' - Type: <class 'str'>
  'sikkim' - Type: <class 'str'>
  'tamil nadu' - Type: <class 'str'>
  'tripura' - Type: <class 'str'>
  'uttar pradesh' - Type: <class 'str'>
  'uttarakhand' - Type: <class 'str'>
  'west bengal' - Type: <class 'str'>
  'telangana' - Type: <class 'str'>
```

```
# DEBUG: Find which state has different names across rounds
print("="*60)
print("DEBUG: FINDING STATE NAME MISMATCH")
print("="*60)

# Get states in NFHS-4 (post=0)
states_nfhs4 = set(df[df['post'] == 0]['state'].unique())
print(f"States in NFHS-4: {len(states_nfhs4)}")
print(f"NFHS-4 states: {sorted(states_nfhs4)}")

# Get states in NFHS-5 (post=1)
```

```
states_nfhs5 = set(df[df['post'] == 1]['state'].unique())
print(f"\nStates in NFHS-5: {len(states_nfhs5)}")
print(f"NFHS-5 states: {sorted(states_nfhs5)}")

# Find mismatches
only_in_nfhs4 = states_nfhs4 - states_nfhs5
only_in_nfhs5 = states_nfhs5 - states_nfhs4

print(f"\nStates ONLY in NFHS-4: {only_in_nfhs4}")
print(f"States ONLY in NFHS-5: {only_in_nfhs5}")

# Show sample counts for mismatched states
print("\n" + "="*60)
print("DETAILS OF MISMATCHED STATES")
print("="*60)

for state in only_in_nfhs4:
    n_count = len(df[(df['state'] == state)])
    print(f"'{state}' appears in {n_count} rows total")
    print(f"  - NFHS-4: {len(df[(df['state'] == state) & (df['post'] == 0)])}")
    print(f"  - NFHS-5: {len(df[(df['state'] == state) & (df['post'] == 1)])}")

for state in only_in_nfhs5:
    n_count = len(df[(df['state'] == state)])
    print(f"'{state}' appears in {n_count} rows total")
    print(f"  - NFHS-4: {len(df[(df['state'] == state) & (df['post'] == 0)])}")
    print(f"  - NFHS-5: {len(df[(df['state'] == state) & (df['post'] == 1)])}")
```

```
============================================================
DEBUG: FINDING STATE NAME MISMATCH
============================================================
States in NFHS-4: 35
NFHS-4 states: ['andaman and nicobar islands', 'andhra pradesh', 'arunachal pradesh', 'assam', 'bihar', 'chandigarh', 'chhattisg

States in NFHS-5: 35
NFHS-5 states: ['andaman and nicobar islands', 'andhra pradesh', 'arunachal pradesh', 'assam', 'bihar', 'chandigarh', 'chhattisg

States ONLY in NFHS-4: set()
States ONLY in NFHS-5: set()

============================================================
DETAILS OF MISMATCHED STATES
============================================================
```

```
# ============================================================
# FIX WEALTH QUINTILE (Convert hv270 to numeric)
# ============================================================

print("="*60)
print("FIXING WEALTH QUINTILE")
print("="*60)

# Map wealth categories to numeric values
wealth_map = {
    'poorest': 1,
    'poorer': 2,
    'middle': 3,
    'richer': 4,
    'richest': 5
}

# Convert hv270 to numeric wealth quintile
df['wealth_quintile'] = df['hv270'].astype(str).str.lower().map(wealth_map)

print(f"Wealth quintile range: {df['wealth_quintile'].min():.0f} to {df['wealth_quintile'].max():.0f}")
print(f"Wealth quintile mean: {df['wealth_quintile'].mean():.2f}")

# Create rich dummy (top 2 quintiles)
df['rich'] = (df['wealth_quintile'] >= 4).astype(int)

print(f"Rich (Q4-Q5) rate: {df['rich'].mean()*100:.1f}%")
```

```
============================================================
FIXING WEALTH QUINTILE
============================================================
Wealth quintile range: 1 to 5
Wealth quintile mean: 2.85
Rich (Q4-Q5) rate: 35.2%
```

```
Start coding or generate with AI.
```

```
# ============================================================
# PART 1: DEFINE TREATMENT (Based on NFHS-4 median)
# ============================================================

print("\n" + "="*60)
print("STEP 1: DEFINING TREATMENT (High Exposure)")
print("="*60)

# Calculate state-level weighted clean fuel for NFHS-4 only
state_nfhs4 = df[df['post'] == 0].groupby('state').apply(
    lambda x: np.average(x['clean_fuel'], weights=x['weight']) * 100,
    include_groups=False
).sort_values()

print(f"Number of states in NFHS-4: {len(state_nfhs4)}")

# Find median
median_value = state_nfhs4.median()

# Define treatment (below median = high exposure)
treatment_states = state_nfhs4[state_nfhs4 < median_value].index.tolist()

# Add treatment indicator
df['high_exposure'] = df['state'].isin(treatment_states).astype(int)

print(f"Median NFHS-4 clean fuel: {median_value:.1f}%")
print(f"Treatment states (below median): {len(treatment_states)}")
print(f"Control states (above median): {len(state_nfhs4) - len(treatment_states)}")
```

```
============================================================
STEP 1: DEFINING TREATMENT (High Exposure)
============================================================
Number of states in NFHS-4: 35
Median NFHS-4 clean fuel: 52.3%
Treatment states (below median): 17
Control states (above median): 18
```

```
# Which states ended up in which group?
state_check = (
    df[df['post'] == 0]
    .groupby('state')
    .apply(lambda x: np.average(x['clean_fuel'], weights=x['weight']) * 100,
           include_groups=False)
    .reset_index()
    .rename(columns={0: 'baseline_clean_pct'})
    .sort_values('baseline_clean_pct')
)

median_val = state_check['baseline_clean_pct'].median()
state_check['high_exposure'] = (state_check['baseline_clean_pct'] < median_val).astype(int)

print("=== ALL STATES SORTED BY BASELINE CLEAN FUEL ===")
print(state_check.to_string(index=False))
print(f"\nMedian: {median_val:.1f}%")
print(f"\nTreatment (high_exposure=1, PMUY targets):")
print(state_check[state_check['high_exposure']==1]['state'].tolist())
print(f"\nControl (high_exposure=0):")
print(state_check[state_check['high_exposure']==0]['state'].tolist())
```

```
=== ALL STATES SORTED BY BASELINE CLEAN FUEL ===
          state  baseline_clean_pct  high_exposure
          bihar           17.760521              1
      jharkhand           18.959144              1
         odisha           19.168095              1
      meghalaya           21.842860              1
   chhattisgarh           22.780439              1
          assam           25.085946              1
    west bengal           27.867793              1
  madhya pradesh           29.595975              1
         tripura           31.867841              1
      rajasthan           31.872670              1
     lakshadweep           31.909577              1
   uttar pradesh           32.749978              1
```

```
                              nagaland          32.804802             1
                        himachal pradesh          36.782035             1
                                 manipur          42.114899             1
                       arunachal pradesh          45.010263             1
                             uttarakhand          51.148394             1
                                 haryana          52.333008             0
                                 gujarat          52.934907             0
                               karnataka          54.919125             0
                                  kerala          57.454658             0
                       jammu and kashmir          57.650000             0
                                  sikkim          59.235211             0
                             maharashtra          60.267984             0
                          andhra pradesh          62.221592             0
              andaman and nicobar islands          63.824074             0
dadra and nagar haveli and daman and diu          64.406358             0
                                  punjab          66.074185             0
                                 mizoram          66.122256             0
                               telangana          67.473712             0
                              tamil nadu          73.230888             0
                                     goa          84.237489             0
                               puducherry          84.962246             0
                               chandigarh          94.532483             0
                                    delhi          98.364886             0

Median: 52.3%

Treatment (high_exposure=1, PMUY targets):
['bihar', 'jharkhand', 'odisha', 'meghalaya', 'chhattisgarh', 'assam', 'west bengal', 'madhya pradesh', 'tripura', 'rajasthan',

Control (high_exposure=0):
['haryana', 'gujarat', 'karnataka', 'kerala', 'jammu and kashmir', 'sikkim', 'maharashtra', 'andhra pradesh', 'andaman and nicob
```

```
# ============================================================
# PART 2: SUMMARY STATISTICS (RAW MEANS)
# ============================================================

print("\n" + "="*80)
print("STEP 2: COMPREHENSIVE SUMMARY STATISTICS (RAW MEANS)")
print("="*80)

# Define groups - VERIFY THESE ARE CORRECT
# post = 0 is NFHS-4 (Pre), post = 1 is NFHS-5 (Post)
# high_exposure = 1 is Treatment (low baseline), high_exposure = 0 is Control (high baseline)
groups = [
    ('Pre-Treatment', 0, 1),   # Pre (NFHS-4), Treatment (high_exposure=1)
    ('Pre-Control', 0, 0),     # Pre (NFHS-4), Control (high_exposure=0)
    ('Post-Treatment', 1, 1),  # Post (NFHS-5), Treatment (high_exposure=1)
    ('Post-Control', 1, 0)     # Post (NFHS-5), Control (high_exposure=0)
]

# Define variables to summarize (NO PERCENTAGE CONVERSION)
variables = {
    'Clean Fuel (0/1)': 'clean_fuel',      # Raw mean = proportion (0-1)
    'Household Size': 'hv009',
    'Head Age': 'hv220',
    'Wealth Quintile (1-5)': 'wealth_quintile',
    'Rural (0/1)': 'rural',               # Raw mean = proportion rural
    'Electricity (0/1)': 'electricity',   # Raw mean = proportion with electricity
    'Female Head (0/1)': 'female_head',   # Raw mean = proportion female-headed
    'Improved Water (0/1)': 'improved_water',
    'Improved Floor (0/1)': 'improved_floor',
    'Piped Water (0/1)': 'piped_water',
    'Rich Q4-Q5 (0/1)': 'rich',
    'Head Higher Edu (0/1)': 'head_higher_edu',
}

# Function to safely convert to numeric
def to_numeric_safe(series):
    return pd.to_numeric(series, errors='coerce')

# Prepare dataframe
df_clean = df.copy()
for var_col in variables.values():
    if var_col in df_clean.columns and var_col is not None:
        df_clean[var_col] = to_numeric_safe(df_clean[var_col])

# Store results
results = {'mean': {}, 'std': {}, 'min': {}, 'max': {}}
```

```
for var_name, var_col in variables.items():
    if var_col is None or var_col not in df_clean.columns:
        print(f"Warning: '{var_name}' column '{var_col}' not found - skipping")
        continue

    for stat_name in ['mean', 'std', 'min', 'max']:
        results[stat_name][var_name] = {}

    for label, post_val, treat_val in groups:
        subset = df_clean[(df_clean['post'] == post_val) & (df_clean['high_exposure'] == treat_val)]

        values = df_clean.loc[subset.index, var_col]
        valid_values = values.dropna()
        valid_weights = subset.loc[valid_values.index, 'weight'] if len(valid_values) > 0 else []

        if len(valid_values) > 0:
            # Weighted mean (RAW - no percentage conversion)
            weighted_mean = np.average(valid_values, weights=valid_weights)
            results['mean'][var_name][label] = weighted_mean

            # Weighted standard deviation
            variance = np.average((valid_values - weighted_mean)**2, weights=valid_weights)
            weighted_std = np.sqrt(variance)
            results['std'][var_name][label] = weighted_std

            # Min and Max
            results['min'][var_name][label] = valid_values.min()
            results['max'][var_name][label] = valid_values.max()
        else:
            for stat_name in ['mean', 'std', 'min', 'max']:
                results[stat_name][var_name][label] = np.nan

# Print combined summary table (RAW MEANS)
print("\nCOMBINED SUMMARY (Mean ± Std Dev) - RAW VALUES")
print("="*100)

combined_data = {'Variable': []}
for label, _, _ in groups:
    combined_data[label] = []

for var_name in variables.keys():
    if var_name not in results['mean']:
        continue
    combined_data['Variable'].append(var_name)
    for label, _, _ in groups:
        mean_val = results['mean'][var_name].get(label, np.nan)
        std_val = results['std'][var_name].get(label, np.nan)
        if pd.isna(mean_val) or pd.isna(std_val):
            formatted = 'N/A'
        else:
            # For binary variables (0/1), show as proportion (e.g., 0.284)
            # For continuous variables, show as is
            if '0/1' in var_name or var_name in ['Rural (0/1)', 'Electricity (0/1)', 'Female Head (0/1)',
                                                 'Improved Water (0/1)', 'Improved Floor (0/1)',
                                                 'Piped Water (0/1)', 'Rich Q4-Q5 (0/1)',
                                                 'Head Higher Edu (0/1)', 'Clean Fuel (0/1)']:
                formatted = f"{mean_val:.3f} ± {std_val:.3f}"
            else:
                formatted = f"{mean_val:.2f} ± {std_val:.2f}"
        combined_data[label].append(formatted)

# Add observations
combined_data['Variable'].append('Observations (N)')
for label, post_val, treat_val in groups:
    subset = df[(df['post'] == post_val) & (df['high_exposure'] == treat_val)]
    combined_data[label].append(f"{len(subset):,}")

combined_table = pd.DataFrame(combined_data)
print(combined_table.to_string(index=False))

# Print key finding (Clean Fuel Rate in PERCENTAGE for interpretation)
print("\n" + "="*80)
print("KEY FINDING - Clean Fuel Rates (Converted to Percentage for Interpretation)")
print("="*80)

pre_treatment = results['mean']['Clean Fuel (0/1)']['Pre-Treatment'] * 100
```

```
post_treatment = results['mean']['Clean Fuel (0/1)']['Post-Treatment'] * 100
pre_control = results['mean']['Clean Fuel (0/1)']['Pre-Control'] * 100
post_control = results['mean']['Clean Fuel (0/1)']['Post-Control'] * 100

print(f"Treatment Group (High Exposure): {pre_treatment:.1f}% → {post_treatment:.1f}% (Change: +{post_treatment - pre_treatment
print(f"Control Group (Low Exposure):   {pre_control:.1f}% → {post_control:.1f}% (Change: {post_control - pre_control:.1f}pp)")
print(f"\nDiD Effect: {(post_treatment - pre_treatment) - (post_control - pre_control):.1f} percentage points")

# Also show raw proportions
print("\n" + "="*80)
print("VERIFICATION - Group Definitions")
print("="*80)
print("Pre = NFHS-4 (post=0)")
print("Post = NFHS-5 (post=1)")
print("Treatment = High Exposure states (below median NFHS-4 clean fuel)")
print("Control = Low Exposure states (above/equal median NFHS-4 clean fuel)")
```

```
================================================================================
STEP 2: COMPREHENSIVE SUMMARY STATISTICS (RAW MEANS)
================================================================================

COMBINED SUMMARY (Mean ± Std Dev) - RAW VALUES
================================================================================================================
             Variable Pre-Treatment   Pre-Control Post-Treatment  Post-Control
     Clean Fuel (0/1) 0.274 ± 0.446 0.630 ± 0.483  0.420 ± 0.494 0.795 ± 0.404
       Household Size   4.98 ± 2.43   4.37 ± 2.06    4.74 ± 2.26   4.11 ± 2.00
             Head Age 47.63 ± 14.19 49.33 ± 13.72  48.68 ± 14.18 51.13 ± 13.81
Wealth Quintile (1-5)   2.49 ± 1.38   3.60 ± 1.22    2.52 ± 1.38   3.54 ± 1.25
          Rural (0/1) 0.755 ± 0.430 0.532 ± 0.499  0.759 ± 0.428 0.557 ± 0.497
    Electricity (0/1) 0.805 ± 0.396 0.971 ± 0.167  0.949 ± 0.221 0.986 ± 0.117
    Female Head (0/1) 0.147 ± 0.354 0.145 ± 0.353  0.168 ± 0.374 0.182 ± 0.386
 Improved Water (0/1) 0.920 ± 0.271 0.931 ± 0.254  0.824 ± 0.381 0.733 ± 0.442
 Improved Floor (0/1) 0.439 ± 0.496 0.772 ± 0.420  0.519 ± 0.500 0.806 ± 0.396
    Piped Water (0/1) 0.154 ± 0.361 0.472 ± 0.499  0.195 ± 0.397 0.494 ± 0.500
     Rich Q4-Q5 (0/1) 0.261 ± 0.439 0.567 ± 0.495  0.267 ± 0.442 0.550 ± 0.498
Head Higher Edu (0/1) 0.458 ± 0.498 0.567 ± 0.495  0.489 ± 0.500 0.577 ± 0.494
     Observations (N)       390,095       210,233        381,199       254,425

================================================================================
KEY FINDING - Clean Fuel Rates (Converted to Percentage for Interpretation)
================================================================================
Treatment Group (High Exposure): 27.4% → 42.0% (Change: +14.6pp)
Control Group (Low Exposure):   63.0% → 79.5% (Change: 16.6pp)

DiD Effect: -2.0 percentage points

================================================================================
VERIFICATION - Group Definitions
================================================================================
Pre = NFHS-4 (post=0)
Post = NFHS-5 (post=1)
Treatment = High Exposure states (below median NFHS-4 clean fuel)
Control = Low Exposure states (above/equal median NFHS-4 clean fuel)
```

```
df.shape
```

```
(1235952, 44)
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# ============================================================
# SUMMARY STATISTICS SECTION — PMUY CLEAN FUEL PROJECT
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import os

os.makedirs("outputs/tables", exist_ok=True)
os.makedirs("outputs/figures", exist_ok=True)

# ------------------------------------------------------------
# Helper function: weighted mean
# ------------------------------------------------------------

def weighted_mean(data, var, weight="weight"):
    temp = data[[var, weight]].dropna()
    if len(temp) == 0:
        return np.nan
    return np.average(temp[var], weights=temp[weight])


# ------------------------------------------------------------
# Helper function: weighted standard deviation
# ------------------------------------------------------------

def weighted_sd(data, var, weight="weight"):
    temp = data[[var, weight]].dropna()
    if len(temp) == 0:
        return np.nan
    mean = np.average(temp[var], weights=temp[weight])
    var_w = np.average((temp[var] - mean) ** 2, weights=temp[weight])
    return np.sqrt(var_w)
```

```
# ============================================================
# TABLE 1: MAIN SUMMARY STATISTICS BY TREATMENT AND PERIOD
# ============================================================

summary_vars = {
    "Clean fuel adoption (%)": "clean_fuel",
    "Household size": "hv009",
    "Head age": "hv220",
    "Wealth quintile": "wealth_quintile",
    "Rural household (%)": "rural",
    "Electricity access (%)": "electricity",
    "Female-headed household (%)": "female_head",
    "Improved water source (%)": "improved_water",
    "Improved floor (%)": "improved_floor",
    "Piped water (%)": "piped_water",
    "Rich household, Q4-Q5 (%)": "rich",
    "Head has secondary+ education (%)": "head_higher_edu"
}

groups = {
    "Pre-Treatment": (0, 1),
    "Pre-Control": (0, 0),
    "Post-Treatment": (1, 1),
    "Post-Control": (1, 0)
}

rows = []

for label, var in summary_vars.items():
    row = {"Variable": label}

    for group_name, (post_val, treat_val) in groups.items():
        subset = df[(df["post"] == post_val) & (df["high_exposure"] == treat_val)].copy()

        subset[var] = pd.to_numeric(subset[var], errors="coerce")

        mean_val = weighted_mean(subset, var)
        sd_val = weighted_sd(subset, var)

        # Convert binary/proportion variables into percentages
        if "(%)" in label:
            mean_val = mean_val * 100
            sd_val = sd_val * 100
            row[group_name] = f"{mean_val:.1f} ({sd_val:.1f})"
        else:
            row[group_name] = f"{mean_val:.2f} ({sd_val:.2f})"

    rows.append(row)
```

```
# Add observation counts
obs_row = {"Variable": "Observations"}
for group_name, (post_val, treat_val) in groups.items():
    subset = df[(df["post"] == post_val) & (df["high_exposure"] == treat_val)]
    obs_row[group_name] = f"{len(subset):,}"

rows.append(obs_row)

summary_table = pd.DataFrame(rows)

print("TABLE 1: Main Summary Statistics")
display(summary_table)

print("""
Interpretation:
Table 1 shows that high-exposure states were systematically different from low-exposure states even before PMUY.
Before the policy, treatment states had lower clean-fuel adoption, lower wealth, lower electricity access,
and a higher rural share. This supports the project motivation: PMUY was especially relevant for states
that were initially more dependent on solid fuels.
""")
```

```
TABLE 1: Main Summary Statistics
```

	Variable	Pre-Treatment	Pre-Control	Post-Treatment	Post-Control
0	Clean fuel adoption (%)	27.4 (44.6)	63.0 (48.3)	42.0 (49.4)	79.5 (40.4)
1	Household size	4.98 (2.43)	4.37 (2.06)	4.74 (2.26)	4.11 (2.00)
2	Head age	47.63 (14.19)	49.33 (13.72)	48.68 (14.18)	51.13 (13.81)
3	Wealth quintile	2.49 (1.38)	3.60 (1.22)	2.52 (1.38)	3.54 (1.25)
4	Rural household (%)	75.5 (43.0)	53.2 (49.9)	75.9 (42.8)	55.7 (49.7)
5	Electricity access (%)	80.5 (39.6)	97.1 (16.7)	94.9 (22.1)	98.6 (11.7)
6	Female-headed household (%)	14.7 (35.4)	14.5 (35.3)	16.8 (37.4)	18.2 (38.6)
7	Improved water source (%)	92.0 (27.1)	93.1 (25.4)	82.4 (38.1)	73.3 (44.2)
8	Improved floor (%)	43.9 (49.6)	77.2 (42.0)	51.9 (50.0)	80.6 (39.6)
9	Piped water (%)	15.4 (36.1)	47.2 (49.9)	19.5 (39.7)	49.4 (50.0)
10	Rich household, Q4-Q5 (%)	26.1 (43.9)	56.7 (49.5)	26.7 (44.2)	55.0 (49.8)
11	Head has secondary+ education (%)	45.8 (49.8)	56.7 (49.5)	48.9 (50.0)	57.7 (49.4)
12	Observations	390,095	210,233	381,199	254,425

```
Interpretation:
Table 1 shows that high-exposure states were systematically different from low-exposure states even before PMUY.
Before the policy, treatment states had lower clean-fuel adoption, lower wealth, lower electricity access,
and a higher rural share. This supports the project motivation: PMUY was especially relevant for states
that were initially more dependent on solid fuels.
```

```
# ============================================================
# TABLE 2: RAW CLEAN FUEL CHANGE AND NAIVE DID
# ============================================================

pre_treat = weighted_mean(df[(df["post"] == 0) & (df["high_exposure"] == 1)], "clean_fuel") * 100
post_treat = weighted_mean(df[(df["post"] == 1) & (df["high_exposure"] == 1)], "clean_fuel") * 100

pre_control = weighted_mean(df[(df["post"] == 0) & (df["high_exposure"] == 0)], "clean_fuel") * 100
post_control = weighted_mean(df[(df["post"] == 1) & (df["high_exposure"] == 0)], "clean_fuel") * 100

treat_change = post_treat - pre_treat
control_change = post_control - pre_control
naive_did = treat_change - control_change

did_table = pd.DataFrame({
    "Group": ["Treatment: High-exposure states", "Control: Low-exposure states", "Naive DiD"],
    "NFHS-4 Clean Fuel (%)": [pre_treat, pre_control, np.nan],
    "NFHS-5 Clean Fuel (%)": [post_treat, post_control, np.nan],
    "Change (percentage points)": [treat_change, control_change, naive_did]
})

did_table = did_table.round(2)
```

```
print("TABLE 2: Raw Treatment-Control Change in Clean Fuel Adoption")
display(did_table)

print(f"""
Interpretation:
Clean-fuel adoption increased in both groups between NFHS-4 and NFHS-5.
In high-exposure states, adoption rose from {pre_treat:.1f}% to {post_treat:.1f}%, a gain of {treat_change:.1f} percentage poir
In low-exposure states, adoption rose from {pre_control:.1f}% to {post_control:.1f}%, a gain of {control_change:.1f} percentage
The raw DiD estimate is {naive_did:.1f} percentage points. This is only descriptive and does not yet control for household cova
""")
```

```
TABLE 2: Raw Treatment-Control Change in Clean Fuel Adoption
```

	Group	NFHS-4 Clean Fuel (%)	NFHS-5 Clean Fuel (%)	Change (percentage points)
0	Treatment: High-exposure states	27.42	41.99	14.57
1	Control: Low-exposure states	62.96	79.52	16.56
2	Naive DiD	NaN	NaN	-2.00

```
Interpretation:
Clean-fuel adoption increased in both groups between NFHS-4 and NFHS-5.
In high-exposure states, adoption rose from 27.4% to 42.0%, a gain of 14.6 percentage points.
In low-exposure states, adoption rose from 63.0% to 79.5%, a gain of 16.6 percentage points.
The raw DiD estimate is -2.0 percentage points. This is only descriptive and does not yet control for household covariates or fi
```

```
# ============================================================
# TABLE 3: BASELINE STATE CLASSIFICATION
# ============================================================

state_baseline = (
    df[df["post"] == 0]
    .groupby("state")
    .apply(lambda x: np.average(x["clean_fuel"], weights=x["weight"]) * 100)
    .reset_index()
)

state_baseline.columns = ["State", "Baseline clean fuel share (%)"]

median_clean = state_baseline["Baseline clean fuel share (%)"].median()

state_baseline["High exposure"] = np.where(
    state_baseline["Baseline clean fuel share (%)"] < median_clean,
    "Treatment",
    "Control"
)

state_baseline = state_baseline.sort_values("Baseline clean fuel share (%)")
state_baseline["Baseline clean fuel share (%)"] = state_baseline["Baseline clean fuel share (%)"].round(2)

print("TABLE 3: State Classification Based on NFHS-4 Clean Fuel Share")
display(state_baseline)

state_baseline.to_csv("outputs/tables/table3_state_classification.csv", index=False)

print(f"Median baseline clean-fuel share: {median_clean:.2f}%")

print("""
Interpretation:
This table shows how treatment status is assigned. States below the NFHS-4 median clean-fuel share are classified
as high-exposure states. These are the states where PMUY was expected to be most relevant because baseline clean-fuel
access was relatively low.
""")
```

```
TABLE 3: State Classification Based on NFHS-4 Clean Fuel Share
/tmp/ipykernel_7758/408140200.py:8: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior i
  .apply(lambda x: np.average(x["clean_fuel"], weights=x["weight"]) * 100)
```

	State	Baseline clean fuel share (%)	High exposure
4	bihar	17.76	Treatment
14	jharkhand	18.96	Treatment
24	odisha	19.17	Treatment
21	meghalaya	21.84	Treatment
6	chhattisgarh	22.78	Treatment
3	assam	25.09	Treatment
34	west bengal	27.87	Treatment
18	madhya pradesh	29.60	Treatment
31	tripura	31.87	Treatment
27	rajasthan	31.87	Treatment
17	lakshadweep	31.91	Treatment
32	uttar pradesh	32.75	Treatment
23	nagaland	32.80	Treatment
12	himachal pradesh	36.78	Treatment
20	manipur	42.11	Treatment
2	arunachal pradesh	45.01	Treatment
33	uttarakhand	51.15	Treatment
11	haryana	52.33	Control
10	gujarat	52.93	Control
15	karnataka	54.92	Control
16	kerala	57.45	Control
13	jammu and kashmir	57.65	Control
28	sikkim	59.24	Control
19	maharashtra	60.27	Control
1	andhra pradesh	62.22	Control
0	andaman and nicobar islands	63.82	Control
7	dadra and nagar haveli and daman and diu	64.41	Control
26	punjab	66.07	Control
22	mizoram	66.12	Control
25	puducherry	84.96	Control

Start coding or generate with AI.

Start coding or generate with AI.

```
# Cell 1 — Install dependencies
!pip install geopandas matplotlib mapclassify -q
!apt-get install -y libspatialindex-dev -q
```

```
Median baseline clean-fuel share: 52.33%
```

Show hidden output

```
Interpretation:
```

Start coding or generate with AI.

```
access was relatively low.
```

```
import os
import requests

REPO_BASE = "https://raw.githubusercontent.com/AnujTiwari/India-State-and-Country-Shapefile-Updated-Jan-2020/master/"

FILES = [
    "India_State_Boundary.shp",
    "India_State_Boundary.dbf",
    "India_State_Boundary.shx",
```

```
    "India_State_Boundary.prj",
]

os.makedirs("shapefiles", exist_ok=True)

for fname in FILES:
    url  = REPO_BASE + fname
    dest = f"shapefiles/{fname}"
    if not os.path.exists(dest):
        print(f"Downloading {fname}...")
        r = requests.get(url)
        if r.status_code == 200:
            with open(dest, "wb") as f:
                f.write(r.content)
            print(f"  ✓ {fname}")
        else:
            print(f"  ✗ Failed ({r.status_code}): {fname}")
    else:
        print(f"  ✓ Already exists: {fname}")
```

```
  ✓ Already exists: India_State_Boundary.shp
  ✓ Already exists: India_State_Boundary.dbf
  ✓ Already exists: India_State_Boundary.shx
  ✓ Already exists: India_State_Boundary.prj
```

```
Start coding or generate with AI.
```

```
import geopandas as gpd
import pandas as pd
import numpy as np

gdf = gpd.read_file("shapefiles/India_State_Boundary.shp")
print("Shapefile CRS:", gdf.crs)
print(f"Number of features: {len(gdf)}")
print("\nColumns:", gdf.columns.tolist())
print("\nState names in shapefile:")
for col in gdf.columns:
    if gdf[col].dtype == object:
        print(f"\n  Column '{col}':")
        print(gdf[col].sort_values().tolist())
```

```
Shapefile CRS: EPSG:3857
Number of features: 37

Columns: ['State_Name', 'geometry']

State names in shapefile:

  Column 'State_Name':
['Andaman & Nicobar', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Chhattishgarh', 'Daman and Diu and
```

```
Start coding or generate with AI.
```

```
# NAME_COL is confirmed as 'State_Name'
NAME_COL = "State_Name"
gdf["state_raw"] = gdf[NAME_COL].astype(str).str.strip()

# Exact mapping based on your shapefile output
SHAPE_TO_DATA = {
    "Andaman & Nicobar"                       : "andaman and nicobar islands",
    "Andhra Pradesh"                          : "andhra pradesh",
    "Arunachal Pradesh"                       : "arunachal pradesh",
    "Assam"                                   : "assam",
    "Bihar"                                   : "bihar",
    "Chandigarh"                              : "chandigarh",
    "Chhattishgarh"                           : "chhattisgarh",   # note typo in shapefile
    "Daman and Diu and Dadra and Nagar Haveli"  : "dadra and nagar haveli and daman and diu",
    "Delhi"                                   : "delhi",
    "Goa"                                     : "goa",
    "Gujarat"                                 : "gujarat",
    "Haryana"                                 : "haryana",
    "Himachal Pradesh"                        : "himachal pradesh",
    "Jammu and Kashmir"                       : "jammu and kashmir",
    "Jharkhand"                               : "jharkhand",
    "Karnataka"                               : "karnataka",
```

```
    "Kerala"                                : "kerala",
    "Ladakh"                                : None,             # not in your data — will show grey
    "Lakshadweep"                           : "lakshadweep",
    "Madhya Pradesh"                        : "madhya pradesh",
    "Maharashtra"                           : "maharashtra",
    "Manipur"                               : "manipur",
    "Meghalaya"                             : "meghalaya",
    "Mizoram"                               : "mizoram",
    "Nagaland"                              : "nagaland",
    "Odisha"                                : "odisha",
    "Puducherry"                            : "puducherry",     # appears twice in shapefile — both map fine
    "Punjab"                                : "punjab",
    "Rajasthan"                             : "rajasthan",
    "Sikkim"                                : "sikkim",
    "Tamilnadu"                             : "tamil nadu",     # no space in shapefile
    "Telengana"                             : "telangana",      # misspelled in shapefile
    "Tripura"                               : "tripura",
    "Uttar Pradesh"                         : "uttar pradesh",
    "Uttarakhand"                           : "uttarakhand",
    "West Bengal"                           : "west bengal",
}

gdf["state"] = gdf["state_raw"].map(SHAPE_TO_DATA)

# Check
unmapped = gdf[gdf["state"].isna() & (gdf["state_raw"] != "Ladakh")]["state_raw"].unique()
if len(unmapped) > 0:
    print(f"⚠ Still unmapped: {unmapped}")
else:
    print("✓ All shapefile states mapped (Ladakh shown grey — not in NFHS data)")

print("\nMapping result:")
print(gdf[["state_raw", "state"]].to_string(index=False))
```

```
✓ All shapefile states mapped (Ladakh shown grey — not in NFHS data)

Mapping result:
                            state_raw                                 state
                      Andaman & Nicobar             andaman and nicobar islands
                             Chandigarh                               chandigarh
Daman and Diu and Dadra and Nagar Haveli dadra and nagar haveli and daman and diu
                                  Delhi                                   delhi
                                Haryana                                 haryana
                              Jharkhand                                jharkhand
                              Karnataka                                karnataka
                                 Kerala                                   kerala
                             Lakshadweep                               lakshadweep
                          Madhya Pradesh                            madhya pradesh
                             Maharashtra                               maharashtra
                                  Odisha                                    odisha
                              Puducherry                                puducherry
                               Tamilnadu                                tamil nadu
                            Chhattishgarh                              chhattisgarh
                                Telengana                                telangana
                           Andhra Pradesh                            andhra pradesh
                              Puducherry                                puducherry
                                     Goa                                       goa
                         Himachal Pradesh                          himachal pradesh
                                   Punjab                                    punjab
                                Rajasthan                                 rajasthan
                                  Gujarat                                   gujarat
                              Uttarakhand                               uttarakhand
                            Uttar Pradesh                             uttar pradesh
                                   Sikkim                                    sikkim
                                    Assam                                     assam
                       Arunachal Pradesh                         arunachal pradesh
                                Nagaland                                  nagaland
                                 Manipur                                   manipur
                                 Mizoram                                   mizoram
                                 Tripura                                   tripura
                               Meghalaya                                 meghalaya
                              West Bengal                               west bengal
                                   Bihar                                     bihar
                                  Ladakh                                      None
                         Jammu and Kashmir                         jammu and kashmir
```

```
Start coding or generate with AI.
```

```
# ============================================================
# Cell 5 — Compute clean fuel rates by state × period
# Uses your existing df with clean_fuel, weight, post, state columns
# ============================================================

# --- Compute weighted clean fuel % per state per period ---
def state_clean_fuel(post_val):
    return (
        df[df["post"] == post_val]
        .groupby("state")
        .apply(
            lambda x: np.average(x["clean_fuel"], weights=x["weight"]) * 100,
            include_groups=False,
        )
        .reset_index()
        .rename(columns={0: "clean_fuel_pct"})
        .assign(post=post_val)
    )

pre_data  = state_clean_fuel(0)   # NFHS-4
post_data = state_clean_fuel(1)   # NFHS-5

# --- Add treatment flag ---
treatment_states = (
    df[df["post"] == 0]
    .groupby("state")
    .apply(lambda x: np.average(x["clean_fuel"], weights=x["weight"]) * 100,
           include_groups=False)
    .pipe(lambda s: s[s < s.median()].index.tolist())
)

pre_data["high_exposure"]  = pre_data["state"].isin(treatment_states).astype(int)
post_data["high_exposure"] = post_data["state"].isin(treatment_states).astype(int)

print("Pre-period (NFHS-4) state data:")
print(pre_data.sort_values("clean_fuel_pct").to_string(index=False))
print("\nPost-period (NFHS-5) state data:")
print(post_data.sort_values("clean_fuel_pct").to_string(index=False))
```

```
Pre-period (NFHS-4) state data:
                                  state  clean_fuel_pct  post  high_exposure
                                  bihar       17.760521     0              1
                              jharkhand       18.959144     0              1
                                 odisha       19.168095     0              1
                              meghalaya       21.842860     0              1
                            chhattisgarh       22.780439     0              1
                                  assam       25.085946     0              1
                             west bengal       27.867793     0              1
                           madhya pradesh       29.595975     0              1
                                  tripura       31.867841     0              1
                                rajasthan       31.872670     0              1
                               lakshadweep       31.909577     0              1
                             uttar pradesh       32.749978     0              1
                                 nagaland       32.804802     0              1
                          himachal pradesh       36.782035     0              1
                                  manipur       42.114899     0              1
                         arunachal pradesh       45.010263     0              1
                               uttarakhand       51.148394     0              1
                                   haryana       52.333008     0              0
                                   gujarat       52.934907     0              0
                                 karnataka       54.919125     0              0
                                    kerala       57.454658     0              0
                         jammu and kashmir       57.650000     0              0
                                    sikkim       59.235211     0              0
                               maharashtra       60.267984     0              0
                             andhra pradesh       62.221592     0              0
              andaman and nicobar islands       63.824074     0              0
dadra and nagar haveli and daman and diu       64.406358     0              0
                                  punjab       66.074185     0              0
                                 mizoram       66.122256     0              0
                               telangana       67.473712     0              0
                              tamil nadu       73.230888     0              0
                                     goa       84.237489     0              0
                              puducherry       84.962246     0              0
                              chandigarh       94.532483     0              0
                                   delhi       98.364886     0              0

Post-period (NFHS-5) state data:
                                  state  clean_fuel_pct  post  high_exposure
                              jharkhand       31.909561     1              1
                            chhattisgarh       33.014583     1              1
```

```
                meghalaya       33.679098     1             1
                   odisha       34.773220     1             1
                    bihar       37.830246     1             1
           madhya pradesh       40.117100     1             1
              west bengal       40.266122     1             1
                rajasthan       41.409595     1             1
                    assam       42.142371     1             1
                 nagaland       43.004266     1             1
                  tripura       45.371247     1             1
            uttar pradesh       49.562531     1             1
         himachal pradesh       51.805215     1             1
        arunachal pradesh       53.190322     1             1
              uttarakhand       59.271437     1             1
              lakshadweep       59.613251     1             1
                  haryana       59.653894     1             0
                  gujarat       67.098872     1             0
```

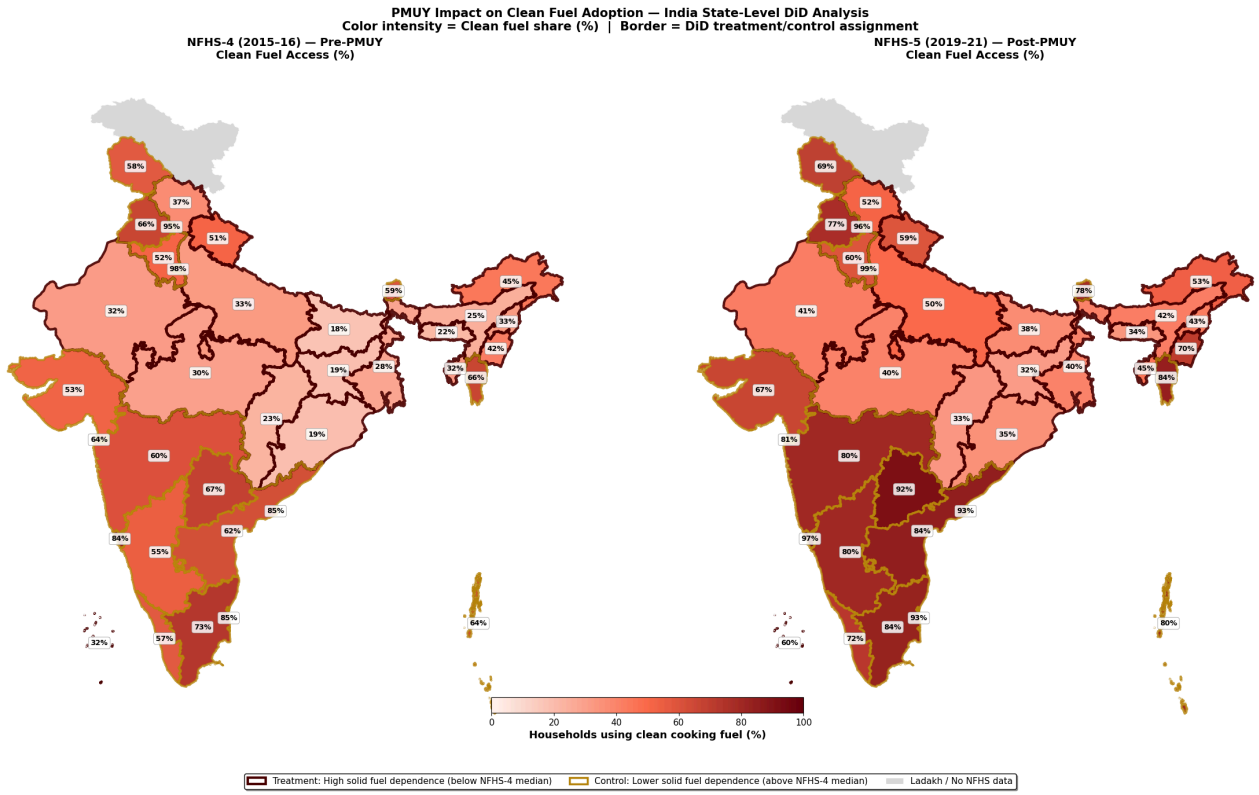
Start coding or generate with AI.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.colors as mcolors
from matplotlib.colors import LinearSegmentedColormap
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")

# ============================================================
# CHECK AND PREPARE DATA
# ============================================================

# Make sure pre_data and post_data have required columns
# Required columns: state, clean_fuel_pct, high_exposure

# Reproject to geographic CRS for correct India shape
gdf_plot = gdf.to_crs(epsg=4326)

# Merge with data
gdf_pre = gdf_plot.merge(pre_data, on="state", how="left")
gdf_post = gdf_plot.merge(post_data, on="state", how="left")

# Ensure clean_fuel_pct is numeric
gdf_pre['clean_fuel_pct'] = pd.to_numeric(gdf_pre['clean_fuel_pct'], errors='coerce')
gdf_post['clean_fuel_pct'] = pd.to_numeric(gdf_post['clean_fuel_pct'], errors='coerce')

# ============================================================
# COLOR SCHEME - UPDATED
# ============================================================

# Red scale for clean fuel adoption
cmap_fuel = LinearSegmentedColormap.from_list(
    "clean_fuel", ["#fff5f0", "#fb6a4a", "#67000d"], N=256
)

# UPDATED COLORS:
TREATMENT_COLOR = "#4a0000"   # Dark maroon for treatment borders
CONTROL_COLOR   = "#b8860b"   # Dark golden/control border (gold)
CONTROL_COLOR_ALT = "#808080"  # Grey alternative for control
MISSING_COLOR   = "#d9d9d9"   # Grey = no data (Ladakh)
VMIN, VMAX = 0, 100

# ============================================================
# PLOT FUNCTION
# ============================================================

def plot_india_map(gdf_data, title, ax):
    """
    Plot India map with choropleth and treatment/control borders
    """
    # Step 1: grey base for all (handles Ladakh and missing)
    gdf_data.plot(
        ax=ax,
        color=MISSING_COLOR,
        edgecolor="white",
        linewidth=0.4,
    )
```

```
        # Step 2: choropleth for states with data
        has_data = gdf_data[gdf_data["clean_fuel_pct"].notna()].copy()
        if len(has_data) > 0:
            has_data.plot(
                column="clean_fuel_pct",
                ax=ax,
                cmap=cmap_fuel,
                vmin=VMIN,
                vmax=VMAX,
                edgecolor="white",
                linewidth=0.4,
                legend=False,
            )

        # Step 3: treatment border — dark maroon, thicker and prominent
        treatment = gdf_data[gdf_data["high_exposure"] == 1]
        if len(treatment) > 0:
            treatment.boundary.plot(
                ax=ax,
                edgecolor=TREATMENT_COLOR,
                linewidth=3.0,  # Thicker for emphasis
                alpha=0.9,
            )

        # Step 4: control border — gold (or grey - choose one)
        control = gdf_data[gdf_data["high_exposure"] == 0]
        if len(control) > 0:
            control.boundary.plot(
                ax=ax,
                edgecolor=CONTROL_COLOR,  # Using gold
                linewidth=2.5,
                alpha=0.8,
            )

        ax.set_title(title, fontsize=14, fontweight="bold", pad=10)
        ax.set_axis_off()

        # Add clean fuel % labels on each state - ENLARGED
        for _, row in gdf_data.iterrows():
            if pd.notna(row.get("clean_fuel_pct")):
                centroid = row.geometry.centroid
                ax.annotate(
                    f"{row['clean_fuel_pct']:.0f}%",
                    xy=(centroid.x, centroid.y),
                    ha="center",
                    va="center",
                    fontsize=9,  # Increased from 6 to 9
                    color="black",
                    fontweight="bold",
                    bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.85, edgecolor="gray", linewidth=0.5)
                )

    # ============================================================
    # DRAW MAPS
    # ============================================================

    fig, axes = plt.subplots(1, 2, figsize=(24, 14))  # Slightly larger for better readability
    fig.patch.set_facecolor("white")

    # Plot pre-period map
    plot_india_map(
        gdf_pre,
        "NFHS-4 (2015–16) — Pre-PMUY\nClean Fuel Access (%)",
        axes[0],
    )

    # Plot post-period map
    plot_india_map(
        gdf_post,
        "NFHS-5 (2019–21) — Post-PMUY\nClean Fuel Access (%)",
        axes[1],
    )

    # ============================================================
    # SHARED COLORBAR
    # ============================================================
```

```
sm = plt.cm.ScalarMappable(
    cmap=cmap_fuel,
    norm=mcolors.Normalize(vmin=VMIN, vmax=VMAX)
)
sm.set_array([])

cbar = fig.colorbar(
    sm, ax=axes,
    orientation="horizontal",
    fraction=0.025,
    pad=0.02,
    shrink=0.55
)
cbar.set_label("Households using clean cooking fuel (%)", fontsize=13, fontweight="bold")
cbar.ax.tick_params(labelsize=11)

# ============================================================
# LEGEND - UPDATED
# ============================================================

treatment_patch = mpatches.Patch(
    facecolor="none",
    edgecolor=TREATMENT_COLOR,
    linewidth=3.0,
    label="Treatment: High solid fuel dependence (below NFHS-4 median)"
)

control_patch = mpatches.Patch(
    facecolor="none",
    edgecolor=CONTROL_COLOR,
    linewidth=2.5,
    label="Control: Lower solid fuel dependence (above NFHS-4 median)"
)

missing_patch = mpatches.Patch(
    facecolor=MISSING_COLOR,
    edgecolor="white",
    label="Ladakh / No NFHS data"
)

fig.legend(
    handles=[treatment_patch, control_patch, missing_patch],
    loc="lower center",
    ncol=3,
    fontsize=11,
    frameon=True,
    edgecolor="black",
    fancybox=True,
    shadow=True,
    bbox_to_anchor=(0.5, 0.01),
)

# ============================================================
# MAIN TITLE
# ============================================================

fig.suptitle(
    "PMUY Impact on Clean Fuel Adoption — India State-Level DiD Analysis\n"
    "Color intensity = Clean fuel share (%)  |  Border = DiD treatment/control assignment",
    fontsize=15,
    fontweight="bold",
    y=0.98,
)

# ============================================================
# SAVE AND DISPLAY
# ============================================================

plt.tight_layout(rect=[0, 0.06, 1, 0.96])
plt.savefig("india_clean_fuel_maps.png", dpi=300, bbox_inches="tight", facecolor="white")
plt.show()

print("="*60)
print("✓ Map saved: india_clean_fuel_maps.png")
print("="*60)

# ============================================================
```

```
# PRINT SUMMARY STATISTICS FOR MAPS
# ============================================================

print("\n📊 MAP DATA SUMMARY:")
print("-"*40)

for period, data, name in [("Pre", gdf_pre, "NFHS-4"), ("Post", gdf_post, "NFHS-5")]:
    valid_data = data[data['clean_fuel_pct'].notna()]
    treatment_data = valid_data[valid_data['high_exposure'] == 1]
    control_data = valid_data[valid_data['high_exposure'] == 0]

    print(f"\n{name} ({period}):")
    print(f"  Total states with data: {len(valid_data)}")
    print(f"  Treatment states (dark maroon border): {len(treatment_data)}")
    print(f"  Control states (gold border): {len(control_data)}")
    print(f"  Missing/Ladakh: {len(data) - len(valid_data)}")

    if len(valid_data) > 0:
        print(f"  Mean clean fuel rate: {valid_data['clean_fuel_pct'].mean():.1f}%")
        print(f"  Range: {valid_data['clean_fuel_pct'].min():.0f}% - {valid_data['clean_fuel_pct'].max():.0f}%")

# ============================================================
# OPTIONAL: UNCOMMENT TO USE GREY INSTEAD OF GOLD
# ============================================================

# To use grey instead of gold for control borders, change:
# CONTROL_COLOR = "#808080"  # Grey
# And update the legend accordingly
```



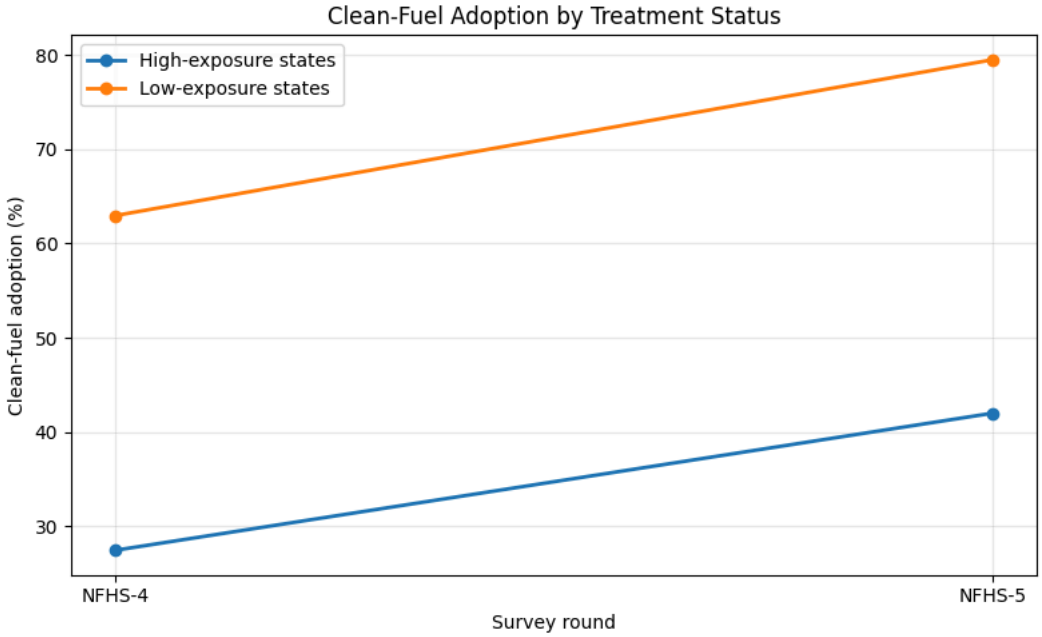

```
============================================================
✓ Map saved: india_clean_fuel_maps.png
============================================================

📊 MAP DATA SUMMARY:
----------------------------------------

NFHS-4 (Pre):
  Total states with data: 36
  Treatment states (dark maroon border): 17
  Control states (gold border): 19
  Missing/Ladakh: 1
  Mean clean fuel rate: 50.7%
  Range: 18% - 98%

NFHS-5 (Post):
  Total states with data: 36
  Treatment states (dark maroon border): 17
  Control states (gold border): 19
  Missing/Ladakh: 1
  Mean clean fuel rate: 64.8%
  Range: 32% - 99%
```

Start coding or generate with AI.

Start coding or generate with AI.

```
# ============================================================
# FIGURE 1: CLEAN FUEL TREND BY TREATMENT STATUS
# ============================================================

trend_table = pd.DataFrame({
    "Period": ["NFHS-4", "NFHS-5"],
    "Treatment": [pre_treat, post_treat],
    "Control": [pre_control, post_control]
})

plt.figure(figsize=(8, 5))

plt.plot(
```

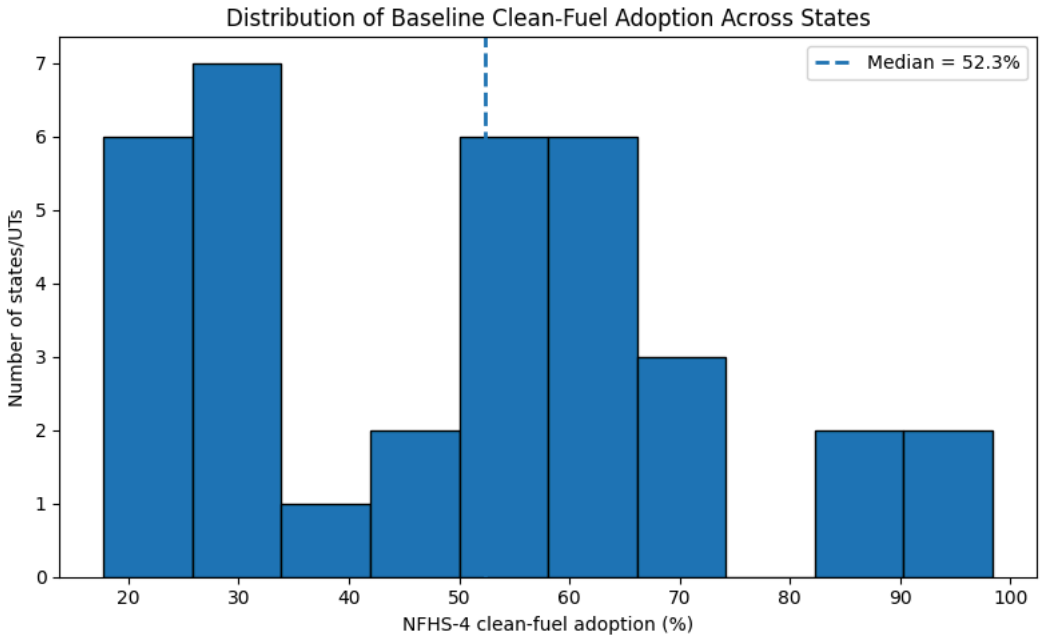
```
    trend_table["Period"],
    trend_table["Treatment"],
    marker="o",
    linewidth=2,
    label="High-exposure states"
)

plt.plot(
    trend_table["Period"],
    trend_table["Control"],
    marker="o",
    linewidth=2,
    label="Low-exposure states"
)

plt.xlabel("Survey round")
plt.ylabel("Clean-fuel adoption (%)")
plt.title("Clean-Fuel Adoption by Treatment Status")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("outputs/figures/figure1_treatment_control_trend.png", dpi=300)
plt.show()

print("""
Interpretation:
The figure shows that clean-fuel adoption increased in both high-exposure and low-exposure states.
However, high-exposure states started from a much lower baseline. This visual motivates the DiD design:
the main question is whether the post-policy change was different across the two groups.
""")
```




```
Interpretation:
The figure shows that clean-fuel adoption increased in both high-exposure and low-exposure states.
However, high-exposure states started from a much lower baseline. This visual motivates the DiD design:
the main question is whether the post-policy change was different across the two groups.
```

```
# ============================================================
# FIGURE 2: HISTOGRAM OF BASELINE CLEAN FUEL SHARE
# ============================================================

plt.figure(figsize=(8, 5))

plt.hist(
    state_baseline["Baseline clean fuel share (%)"],
    bins=10,
    edgecolor="black"
)
```

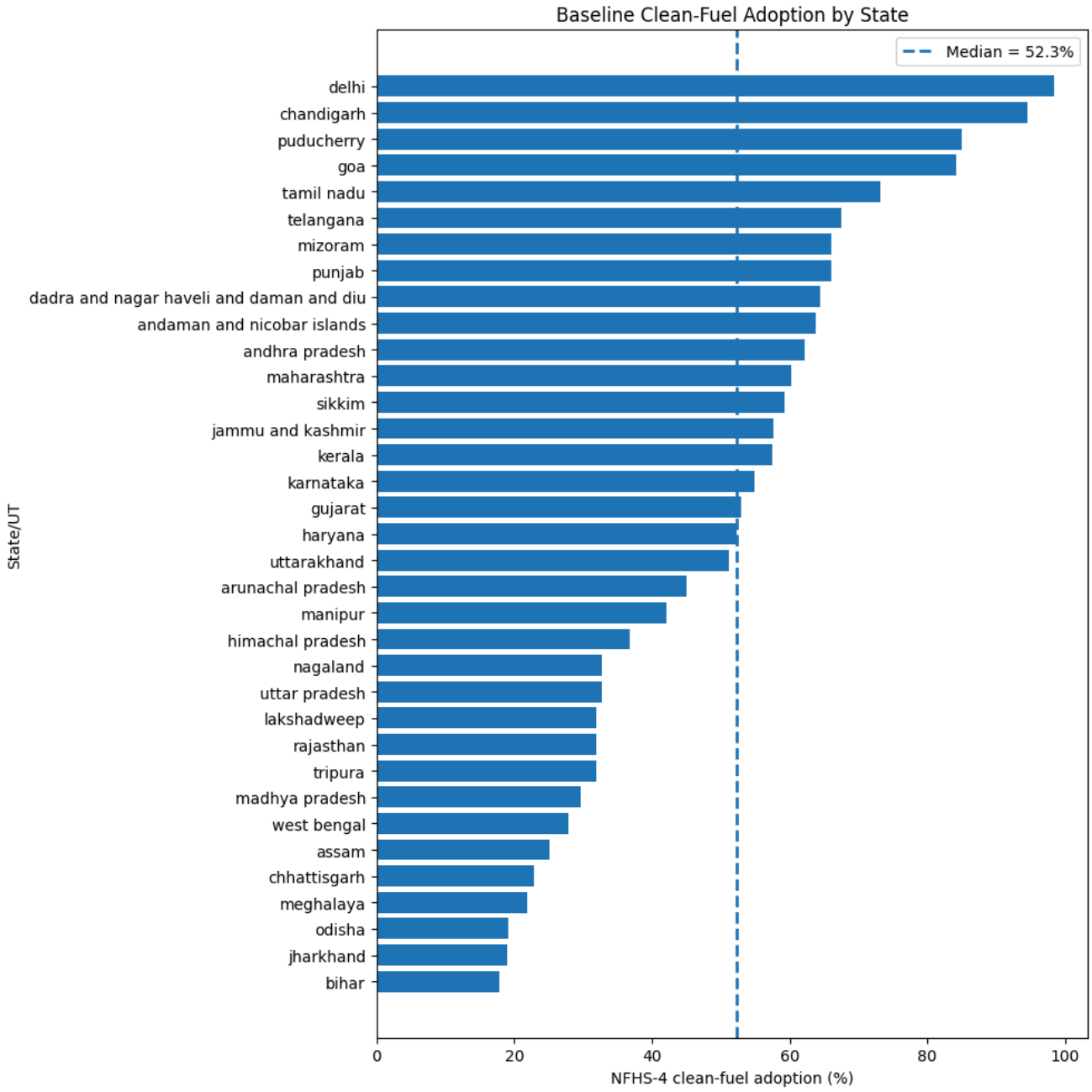
```
plt.axvline(
    median_clean,
    linestyle="--",
    linewidth=2,
    label=f"Median = {median_clean:.1f}%"
)

plt.xlabel("NFHS-4 clean-fuel adoption (%)")
plt.ylabel("Number of states/UTs")
plt.title("Distribution of Baseline Clean-Fuel Adoption Across States")
plt.legend()

plt.tight_layout()
plt.savefig("outputs/figures/figure2_baseline_histogram.png", dpi=300)
plt.show()

print("""
Interpretation:
This histogram shows large cross-state variation in baseline clean-fuel access.
The median line shows the cutoff used to define high-exposure states.
This supports the idea that PMUY was operating in a highly unequal starting environment.
""")
```




```
Interpretation:
This histogram shows large cross-state variation in baseline clean-fuel access.
The median line shows the cutoff used to define high-exposure states.
This supports the idea that PMUY was operating in a highly unequal starting environment.
```

```
# ============================================================
# FIGURE 3: STATE-WISE BASELINE CLEAN FUEL SHARE
# ============================================================

state_plot = state_baseline.sort_values("Baseline clean fuel share (%)")

plt.figure(figsize=(10, 10))

plt.barh(
    state_plot["State"],
    state_plot["Baseline clean fuel share (%)"]
)

plt.axvline(
    median_clean,
    linestyle="--",
    linewidth=2,
    label=f"Median = {median_clean:.1f}%"
)

plt.xlabel("NFHS-4 clean-fuel adoption (%)")
```

```
plt.ylabel("State/UT")
plt.title("Baseline Clean-Fuel Adoption by State")
plt.legend()

plt.tight_layout()
plt.savefig("outputs/figures/figure3_state_baseline_barplot.png", dpi=300)
plt.show()

print("""
Interpretation:
The bar chart highlights which states had the lowest clean-fuel access before PMUY.
States such as Bihar, Jharkhand, Odisha, Chhattisgarh, and Assam appear among the lowest baseline adopters.
This gives a clear policy justification for focusing on initially low-access states.
""")
```




```
Interpretation:
The bar chart highlights which states had the lowest clean-fuel access before PMUY.
States such as Bihar, Jharkhand, Odisha, Chhattisgarh, and Assam appear among the lowest baseline adopters.
This gives a clear policy justification for focusing on initially low-access states.
```

```
# ============================================================
# TABLE 4: CLEAN FUEL ADOPTION BY RURAL/URBAN STATUS
# ============================================================
```

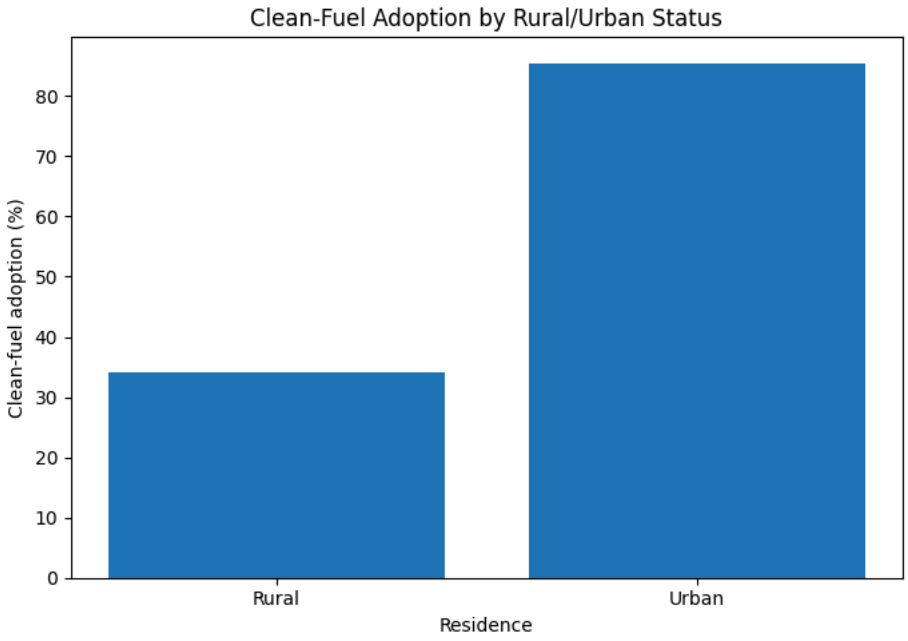
```
rural_table = []

for rural_val, rural_label in [(1, "Rural"), (0, "Urban")]:
    subset = df[df["rural"] == rural_val]
    clean_rate = weighted_mean(subset, "clean_fuel") * 100
    n = len(subset)

    rural_table.append({
        "Residence": rural_label,
        "Clean fuel adoption (%)": round(clean_rate, 2),
        "Observations": f"{n:,}"
    })

rural_table = pd.DataFrame(rural_table)

print("TABLE 4: Clean Fuel Adoption by Rural/Urban Status")
display(rural_table)

rural_rate = rural_table.loc[rural_table["Residence"] == "Rural", "Clean fuel adoption (%)"].iloc[0]
urban_rate = rural_table.loc[rural_table["Residence"] == "Urban", "Clean fuel adoption (%)"].iloc[0]

print(f"""
Interpretation:
Clean-fuel adoption is much lower among rural households than urban households.
The rural clean-fuel adoption rate is {rural_rate:.1f}%, compared with {urban_rate:.1f}% in urban areas.
This is important because PMUY was especially targeted toward poor and rural households.
""")
```

```
TABLE 4: Clean Fuel Adoption by Rural/Urban Status
```

	Residence	Clean fuel adoption (%)	Observations
0	Rural	34.08	900,949
1	Urban	85.49	335,003

```
Interpretation:
Clean-fuel adoption is much lower among rural households than urban households.
The rural clean-fuel adoption rate is 34.1%, compared with 85.5% in urban areas.
This is important because PMUY was especially targeted toward poor and rural households.
```

```
# ============================================================
# FIGURE 4: CLEAN FUEL ADOPTION BY RURAL/URBAN STATUS
# ============================================================

plt.figure(figsize=(7, 5))

plt.bar(
    rural_table["Residence"],
    rural_table["Clean fuel adoption (%)"]
)

plt.ylabel("Clean-fuel adoption (%)")
plt.xlabel("Residence")
plt.title("Clean-Fuel Adoption by Rural/Urban Status")

plt.tight_layout()
plt.savefig("outputs/figures/figure4_rural_urban_clean_fuel.png", dpi=300)
plt.show()
```



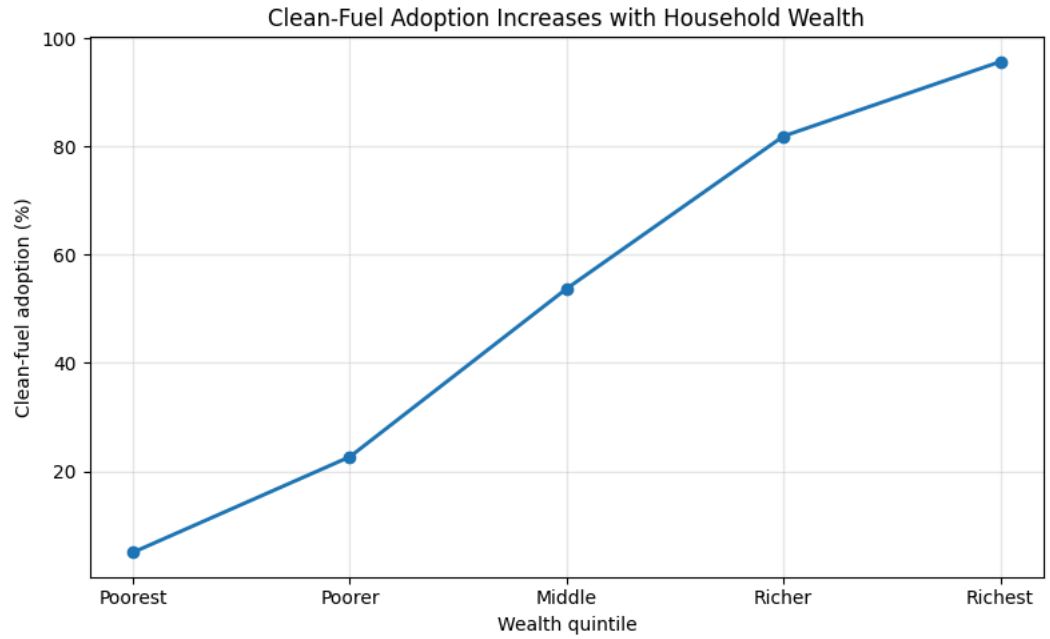

```
# ============================================================
# TABLE 5: CLEAN FUEL ADOPTION BY WEALTH QUINTILE
# ============================================================

wealth_rows = []

wealth_labels = {
    1: "Poorest",
    2: "Poorer",
    3: "Middle",
    4: "Richer",
    5: "Richest"
}

for q in sorted(df["wealth_quintile"].dropna().unique()):
    subset = df[df["wealth_quintile"] == q]
    clean_rate = weighted_mean(subset, "clean_fuel") * 100
    n = len(subset)

    wealth_rows.append({
        "Wealth quintile": wealth_labels.get(int(q), q),
        "Clean fuel adoption (%)": round(clean_rate, 2),
        "Observations": f"{n:,}"
    })

wealth_table = pd.DataFrame(wealth_rows)

print("TABLE 5: Clean Fuel Adoption by Wealth Quintile")
display(wealth_table)

print("""
Interpretation:
Clean-fuel adoption rises strongly with household wealth.
This confirms that clean cooking is closely linked to socioeconomic status.
It also supports the policy motivation for PMUY, since poorer households were less likely to use clean cooking fuel before and
""")
```

```
TABLE 5: Clean Fuel Adoption by Wealth Quintile
```

	Wealth quintile	Clean fuel adoption (%)	Observations
0	Poorest	4.99	279,813
1	Poorer	22.66	270,409
2	Middle	53.75	250,417
3	Richer	81.94	227,169
4	Richest	95.71	208,144

```
Interpretation:
Clean-fuel adoption rises strongly with household wealth.
This confirms that clean cooking is closely linked to socioeconomic status.
It also supports the policy motivation for PMUY, since poorer households were less likely to use clean cooking fuel before and a
```

```
# ============================================================
# FIGURE 5: CLEAN FUEL ADOPTION BY WEALTH QUINTILE
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(
    wealth_table["Wealth quintile"],
    wealth_table["Clean fuel adoption (%)"],
    marker="o",
    linewidth=2
)

plt.xlabel("Wealth quintile")
plt.ylabel("Clean-fuel adoption (%)")
plt.title("Clean-Fuel Adoption Increases with Household Wealth")
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("outputs/figures/figure5_wealth_gradient.png", dpi=300)
plt.show()
```




```
# ============================================================
# TABLE 8: CORRELATION MATRIX
# ============================================================

corr_vars = [
    "clean_fuel",
    "rural",
    "electricity",
    "female_head",
```

```
    "wealth_quintile",
    "improved_water",
    "improved_floor",
    "piped_water",
    "head_higher_edu"
]

corr_df = df[corr_vars].apply(pd.to_numeric, errors="coerce")

corr_matrix = corr_df.corr().round(2)

print("TABLE 8: Correlation Matrix")
display(corr_matrix)

print("""
Interpretation:
The correlation matrix shows how clean-fuel adoption is associated with household characteristics.
Positive correlations with wealth, electricity, improved floor, piped water, and education suggest that clean cooking is closel
with broader household socioeconomic status.
""")
```

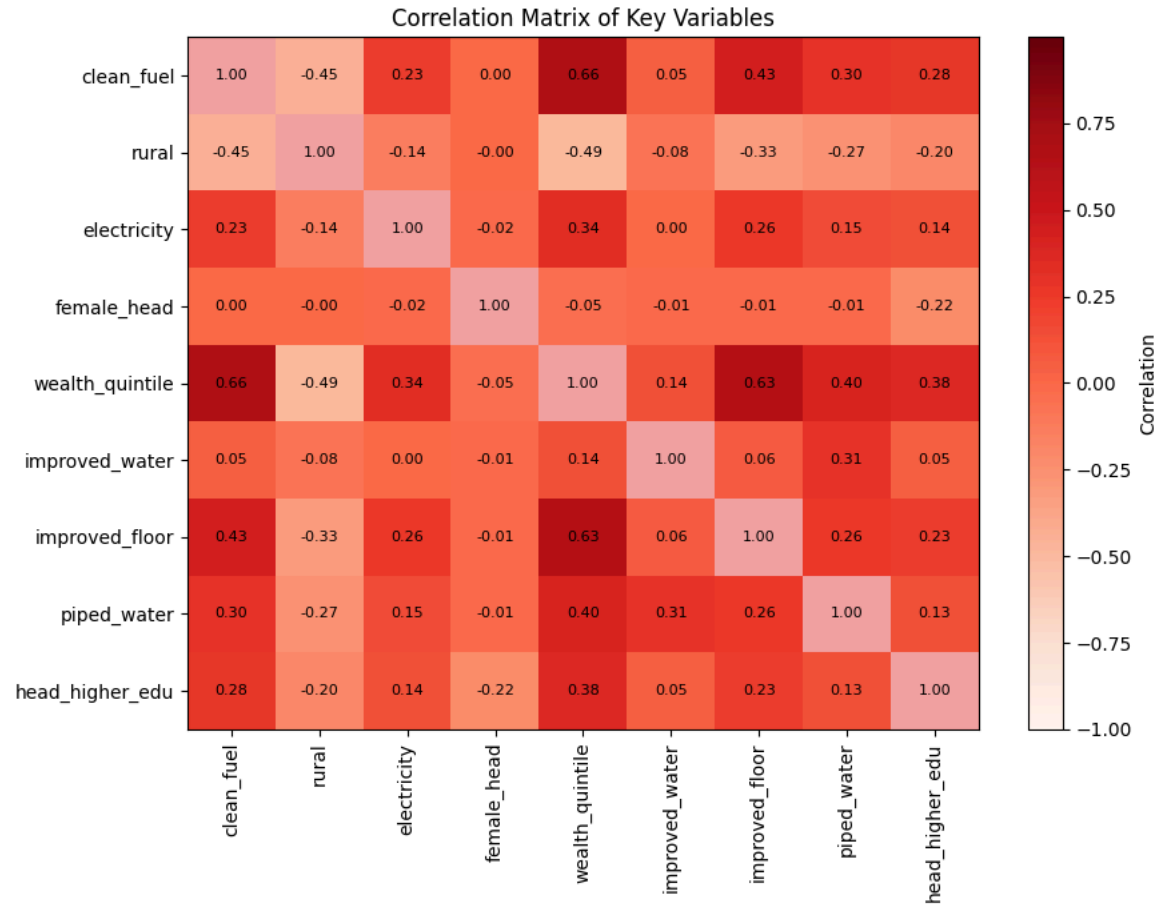
```
TABLE 8: Correlation Matrix
```

	clean_fuel	rural	electricity	female_head	wealth_quintile	improved_water	improved_floor	piped_water	head_
clean_fuel	1.00	-0.45	0.23	0.00	0.66	0.05	0.43	0.30	
rural	-0.45	1.00	-0.14	-0.00	-0.49	-0.08	-0.33	-0.27	
electricity	0.23	-0.14	1.00	-0.02	0.34	0.00	0.26	0.15	
female_head	0.00	-0.00	-0.02	1.00	-0.05	-0.01	-0.01	-0.01	
wealth_quintile	0.66	-0.49	0.34	-0.05	1.00	0.14	0.63	0.40	
improved_water	0.05	-0.08	0.00	-0.01	0.14	1.00	0.06	0.31	
improved_floor	0.43	-0.33	0.26	-0.01	0.63	0.06	1.00	0.26	
piped_water	0.30	-0.27	0.15	-0.01	0.40	0.31	0.26	1.00	
head_higher_edu	0.28	-0.20	0.14	-0.22	0.38	0.05	0.23	0.13	

```
Interpretation:
The correlation matrix shows how clean-fuel adoption is associated with household characteristics.
Positive correlations with wealth, electricity, improved floor, piped water, and education suggest that clean cooking is closely
with broader household socioeconomic status.
```

```
# ============================================================
# FIGURE 6: CORRELATION HEATMAP (SHADES OF RED)
# ============================================================

plt.figure(figsize=(9, 7))

# Use red color palette
heatmap = plt.imshow(
    corr_matrix,
    aspect="auto",
    cmap="Reds",      # Shades of red
    vmin=-1,
    vmax=1
)

# Make ONLY the 1.00 diagonal blocks slightly lighter
heatmap.cmap.set_over('#f4a3a3')
heatmap.set_clim(-1, 0.999)

# Add color bar
cbar = plt.colorbar(heatmap)
cbar.set_label("Correlation")

# Axis labels
plt.xticks(
    range(len(corr_matrix.columns)),
    corr_matrix.columns,
    rotation=90
)

plt.yticks(
```

```
    range(len(corr_matrix.index)),
    corr_matrix.index
)

# Add correlation values inside boxes
for i in range(len(corr_matrix.index)):
    for j in range(len(corr_matrix.columns)):
        plt.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha="center",
            va="center",
            fontsize=8,
            color="black"
        )

plt.title("Correlation Matrix of Key Variables")

plt.tight_layout()

plt.savefig(
    "outputs/figures/figure6_correlation_heatmap_red.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



Start coding or generate with AI.

```
import numpy as np
import pandas as pd
import statsmodels.formula.api as smf
from scipy import stats

print("=" * 80)
print("IMPROVED TWFE DiD REGRESSION")
```

```
print("=" * 80)

# Helper function to ensure robust results params/pvalues/conf_int are pandas objects
def _get_pandas_robust_results(robust_results_obj, original_model):
    """
    Extracts params, pvalues, and conf_int from a statsmodels RobustCovResults
    object and returns them as pandas Series/DataFrame.
    """
    param_names = original_model.params.index

    params = pd.Series(robust_results_obj.params, index=param_names)
    pvalues = pd.Series(robust_results_obj.pvalues, index=param_names)
    conf_int = pd.DataFrame(robust_results_obj.conf_int(), index=param_names)

    return params, pvalues, conf_int

# ============================================================
# RE-LOAD DATA AND RE-APPLY NECESSARY TRANSFORMATIONS
# ============================================================

file_path = "/content/drive/MyDrive/pmuy_data_compressed.csv.gz"
df = pd.read_csv(file_path)

# Ensure 'post' is numeric (0 or 1)
df['post'] = pd.to_numeric(df['post'], errors='coerce')

# Apply weights
df['weight'] = df['hv005'] / 1_000_000

# Apply clean fuel definition
CLEAN_FUELS = ['electricity', 'lpg, natural gas', 'biogas']
df = df[~df['hv226'].isin(['no food cooked in house'])]
df['clean_fuel'] = df['hv226'].isin(CLEAN_FUELS).astype(int)
df = df[df['clean_fuel'].notna()]

# Re-create high_exposure (treatment) flag
state_nfhs4 = df[df['post'] == 0].groupby('state').apply(
    lambda x: np.average(x['clean_fuel'], weights=x['weight']) * 100,
    include_groups=False
).sort_values()
median_value = state_nfhs4.median()
treatment_states = state_nfhs4[state_nfhs4 < median_value].index.tolist()
df['high_exposure'] = df['state'].isin(treatment_states).astype(int)

# Re-create control variables from previous cell
if 'hv025' in df.columns:
    df['rural'] = df['hv025'].str.lower().str.strip().map({'rural': 1, 'urban': 0}).fillna(0).astype(int)
if 'hv206' in df.columns:
    df['electricity'] = df['hv206'].str.lower().str.strip().map({'yes': 1, 'no': 0}).fillna(0).astype(int)
if 'hv219' in df.columns:
    df['female_head'] = df['hv219'].str.lower().str.strip().map({'female': 1, 'male': 0}).fillna(0).astype(int)
if 'hv201' in df.columns:
    improved_water_sources = ['piped into dwelling', 'piped to yard/plot', 'public tap/standpipe', 'tube well or borehole', 'pr
    df['improved_water'] = df['hv201'].str.lower().str.strip().isin(improved_water_sources).astype(int)
if 'hv213' in df.columns:
    unimproved_floors = ['mud/clay/earth', 'dung', 'sand', 'raw wood planks', 'palm, bamboo', 'stone']
    df['improved_floor'] = (~df['hv213'].str.lower().str.strip().isin(unimproved_floors)).astype(int)
if 'hv106_01' in df.columns:
    df['head_edu'] = df['hv106_01'].str.lower().str.strip()
    df['head_higher_edu'] = df['head_edu'].isin(['secondary', 'higher']).astype(int)
if 'hv201' in df.columns:
    piped_sources = ['piped into dwelling', 'piped to yard/plot']
    df['piped_water'] = df['hv201'].str.lower().str.strip().isin(piped_sources).astype(int)
if 'hv270' in df.columns:
    wealth_map = {'poorest': 1, 'poorer': 2, 'middle': 3, 'richer': 4, 'richest': 5}
    if df['hv270'].dtype == 'object':
        df['wealth_quintile'] = df['hv270'].str.lower().str.strip().map(wealth_map)
    else:
        df['wealth_quintile'] = pd.to_numeric(df['hv270'], errors='coerce')
    df['rich'] = (df['wealth_quintile'] >= 4).astype(int)

# ============================================================
# STEP 1: PREPARE DATA
# ============================================================

# Ensure all variables are numeric
numeric_cols = ["clean_fuel", "post", "high_exposure", "weight",
```

```
                "wealth_quintile", "hv009", "hv220", "rural",
                "electricity", "female_head", "improved_water",
                "improved_floor", "piped_water", "rich", "head_higher_edu"]

for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

# Ensure state is string
df["state"] = df["state"].astype(str).str.lower().str.strip()

# Create interaction term
df["did_interaction"] = df["post"] * df["high_exposure"]

# Drop missing values
needed_cols = ["clean_fuel", "post", "high_exposure", "did_interaction",
               "state", "weight"] + [c for c in numeric_cols if c in df.columns]

# Ensure needed_cols has only unique names to prevent potential issues, though not the root cause here
needed_cols = list(np.unique(needed_cols))

# Reset index to ensure uniqueness after dropna, which is causing the ValueError
df_model = df[needed_cols].dropna().reset_index(drop=True).copy()

print(f"\nSample size: {len(df_model):,}")
print(f"States: {df_model['state'].nunique()}")
print(f"Treatment states: {df_model[df_model['high_exposure']==1]['state'].nunique()}")
print(f"Control states: {df_model[df_model['high_exposure']==0]['state'].nunique()}")



controls = ["rural", "electricity", "female_head", "improved_water",
            "improved_floor", "piped_water", "rich", "head_higher_edu",
            "wealth_quintile", "hv009", "hv220"]

controls_str = " + ".join(controls)



# ============================================================
 # Adding State-Specific Time Trends
# (Controls for differential pre-trends across states)
# ============================================================

print("\n--- MODEL 2: TWFE with State-Specific Linear Time Trends ---")

# The manual creation of trend terms (state_dummies, sanitized_state_dummy_cols, etc.)
# is replaced by a more direct patsy formula `C(state):post`.
# `C(state)` handles state fixed effects.
# `C(state):post` adds interactions of each state dummy with the 'post' variable,
# which represent the state-specific linear time trends.
formula2 = f"clean_fuel ~ post + high_exposure + did_interaction + C(state) + C(state):post + {controls_str}"

# DEBUG: Print the formula to inspect it
print(f"  Model 2 Formula: {formula2}")

model2 = smf.wls(formula2, data=df_model, weights=df_model["weight"]).fit()
model2_cl = model2.get_robustcov_results(cov_type="cluster", groups=df_model["state"])
model2_params, model2_pvalues, model2_conf_int = _get_pandas_robust_results(model2_cl, model2)

# Safely extract DiD coefficient for Model 2
if "did_interaction" in model2_params.index:
    did_coef2 = model2_params["did_interaction"] * 100
    ci2_lower, ci2_upper = model2_conf_int.loc["did_interaction"] * 100
    p2 = model2_pvalues["did_interaction"]
else:
    print("  Warning: 'did_interaction' term not found in Model 2 parameters.")
    did_coef2 = np.nan
    ci2_lower = np.nan
    ci2_upper = np.nan
    p2 = np.nan

print(f"  DiD Coefficient: {did_coef2:.3f} pp")
print(f"  95% CI: [{ci2_lower:.3f}, {ci2_upper:.3f}]")
print(f"  P-value: {p2:.4f}")
print(f"  CI excludes zero: {ci2_lower > 0 or ci2_upper < 0}")
```

```
================================================================================
IMPROVED TWFE DiD REGRESSION
================================================================================

Sample size: 1,235,703
States: 35
Treatment states: 17
Control states: 18

--- MODEL 2: TWFE with State-Specific Linear Time Trends ---
  Model 2 Formula: clean_fuel ~ post + high_exposure + did_interaction + C(state) + C(state):post + rural + electricity + female
  DiD Coefficient: -7.682 pp
  95% CI: [-7.977, -7.388]
  P-value: 0.0000
  CI excludes zero: True
```

```
Start coding or generate with AI.
```

```
# ============================================================
# 1. INSTALL AND IMPORT PACKAGES
# ============================================================

!pip install pyreadstat -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pyreadstat
import os

os.makedirs("outputs", exist_ok=True)
os.makedirs("outputs/figures", exist_ok=True)
os.makedirs("outputs/tables", exist_ok=True)
```

```
# ============================================================
# 2. MOUNT GOOGLE DRIVE
# ============================================================

from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
# ============================================================
# 3. LOAD NFHS COMBINED .DTA FILE
# ============================================================

file_path = "/content/drive/MyDrive/NFHS_combined.dta"

df, meta = pyreadstat.read_dta(file_path, apply_value_formats=True)

print(df.shape)
df.head()
```

```
(793652, 19)
```

	hhid	hv005	hv009	hv024	hv025	hv201	hv204	hv206	hv219	hv220	hv106_01	hv226	sh34	sh36	state	state_id	su
0	2 1 3	2205947	4	andhra pradesh	rural	public open well	15	no	male	46	no education	wood	hindu	scheduled caste	andhra pradesh	andhra pradesh	
1	2 1 12	2205947	5	andhra pradesh	rural	public open well	15	no	male	35	no education	wood	hindu	scheduled caste	andhra pradesh	andhra pradesh	
2	2 1 21	2205947	3	andhra pradesh	rural	public open well	10	no	male	38	no education	wood	hindu	scheduled tribe	andhra pradesh	andhra pradesh	
3	2 1 30	2205947	2	andhra pradesh	rural	public open well	10	no	male	55	no education	wood	hindu	scheduled caste	andhra pradesh	andhra pradesh	
4	2 1 39	2205947	9	andhra pradesh	rural	public open well	10	no	male	85	no education	wood	hindu	scheduled caste	andhra pradesh	andhra pradesh	

```
# ============================================================
# 4. CHECK VARIABLES
# ============================================================

print(df.columns.tolist())
print(df.dtypes)

print("\nSurvey rounds:")
print(df["survey"].value_counts().sort_index())

print("\nStates:")
print(sorted(df["state"].dropna().unique()))
```

```
['hhid', 'hv005', 'hv009', 'hv024', 'hv025', 'hv201', 'hv204', 'hv206', 'hv219', 'hv220', 'hv106_01', 'hv226', 'sh34', 'sh36', '
hhid          object
hv005          int64
hv009          int64
hv024       category
hv025       category
hv201       category
hv204       category
hv206       category
hv219       category
hv220       category
hv106_01    category
hv226       category
sh34        category
sh36        category
state         object
state_id    category
survey       float64
hv213       category
hv270       category
dtype: object

Survey rounds:
survey
2.0     92486
3.0    109041
4.0    592125
Name: count, dtype: int64

States:
['andhra pradesh', 'arunachal pradesh', 'assam', 'bihar', 'delhi', 'goa', 'gujarat', 'haryana', 'himachal pradesh', 'jammu and k
```

```
# ============================================================
# 5. CLEAN STATE NAMES
# ============================================================

df["state"] = df["state"].astype(str).str.lower().str.strip()

print(sorted(df["state"].unique()))
```

```
['andhra pradesh', 'arunachal pradesh', 'assam', 'bihar', 'delhi', 'goa', 'gujarat', 'haryana', 'himachal pradesh', 'jammu and k
```

```
# ============================================================
# 6. CREATE TREATMENT GROUP: HIGH EXPOSURE STATES
# ============================================================

treatment_states = [
    "bihar",
    "jharkhand",
    "odisha",
    "meghalaya",
    "chhattisgarh",
    "assam",
    "west bengal",
    "madhya pradesh",
    "tripura",
    "rajasthan",
    "lakshadweep",
    "uttar pradesh",
    "nagaland",
    "himachal pradesh",
    "manipur",
    "arunachal pradesh",
    "uttarakhand"
]

control_states = [
    "haryana",
    "gujarat",
    "karnataka",
    "kerala",
    "jammu and kashmir",
    "sikkim",
    "maharashtra",
    "andhra pradesh",
    "andaman and nicobar islands",
    "dadra and nagar haveli and daman and diu",
    "punjab",
    "mizoram",
    "telangana",
    "tamil nadu",
    "goa",
    "puducherry",
    "chandigarh",
    "delhi"
]

df["high_exposure"] = np.where(df["state"].isin(treatment_states), 1, 0)

print(df["high_exposure"].value_counts())

print("\nTreatment states in data:")
print(sorted(df.loc[df["high_exposure"] == 1, "state"].unique()))

print("\nControl states in data:")
print(sorted(df.loc[df["high_exposure"] == 0, "state"].unique()))
```

```
high_exposure
1    499172
0    294480
Name: count, dtype: int64

Treatment states in data:
['arunachal pradesh', 'assam', 'bihar', 'himachal pradesh', 'madhya pradesh', 'manipur', 'meghalaya', 'nagaland', 'odisha', 'raj

Control states in data:
['andhra pradesh', 'delhi', 'goa', 'gujarat', 'haryana', 'jammu and kashmir', 'karnataka', 'kerala', 'maharashtra', 'mizoram', '
```

```
# ============================================================
# 7. CREATE WEIGHT VARIABLE
# ============================================================

df["weight"] = df["hv005"] / 1_000_000

print(df["weight"].describe())
```

```
count    793652.000000
mean          1.009711
std           1.047153
```

```
min           0.000884
25%           0.379036
50%           0.771679
75%           1.342960
max          38.954801
Name: weight, dtype: float64
```

```
# ============================================================
# 8. CHECK COOKING FUEL CATEGORIES
# ============================================================

df["fuel"] = df["hv226"].astype(str).str.lower().str.strip()

print(df["fuel"].value_counts(dropna=False))
```

```
fuel
liquid petroleum gas    355451
crop residues           253512
wood                     60142
11                       50247
10                       16480
kerosene                 16379
bio-gas                  13395
charcoal                 11163
coal/coke/lignite         5160
electricity               4862
dung cakes                4822
95                        1099
other                      889
99                          26
nan                         25
Name: count, dtype: int64
```

```
# ============================================================
# 9. CREATE CLEAN FUEL VARIABLE (CORRECTED FOR NFHS-2/3/4)
# ============================================================

df["fuel"] = df["hv226"].astype(str).str.lower().str.strip()

print(df["fuel"].value_counts(dropna=False))

# Define clean fuels
clean_fuels = [
    "liquid petroleum gas",
    "electricity",
    "bio-gas"
]

# Drop households with invalid cooking information
drop_values = ["10", "11", "95", "99"]

df = df[~df["fuel"].isin(drop_values)].copy()

# Create clean fuel dummy
df["clean_fuel"] = df["fuel"].isin(clean_fuels).astype(int)

print("\nClean fuel distribution:")
print(df["clean_fuel"].value_counts())

print(f"\nClean fuel adoption rate: {df['clean_fuel'].mean()*100:.2f}%")
```

```
fuel
liquid petroleum gas    355451
crop residues           253512
wood                     60142
11                       50247
10                       16480
kerosene                 16379
bio-gas                  13395
charcoal                 11163
coal/coke/lignite         5160
electricity               4862
dung cakes                4822
95                        1099
other                      889
99                          26
nan                         25
Name: count, dtype: int64

Clean fuel distribution:
```

```
clean_fuel
1    373708
0    352092
Name: count, dtype: int64

Clean fuel adoption rate: 51.49%
```

```
# ============================================================
# 10. KEEP ONLY PRE-POLICY ROUNDS: NFHS-2, NFHS-3, NFHS-4
# ============================================================

pre_df = df[df["survey"].isin([2, 3, 4])].copy()

print(pre_df["survey"].value_counts().sort_index())
print(pre_df.shape)
```

```
survey
2.0     92486
3.0     99165
4.0    534149
Name: count, dtype: int64
(725800, 23)
```

```
# ============================================================
# 11. HELPER FUNCTION FOR WEIGHTED MEAN
# ============================================================

def weighted_mean(x, value_col, weight_col):
    temp = x[[value_col, weight_col]].dropna()
    if len(temp) == 0:
        return np.nan
    return np.average(temp[value_col], weights=temp[weight_col])
```

```
# ============================================================
# 12. TABLE: CLEAN FUEL ADOPTION BY SURVEY AND GROUP
# ============================================================

trend_table = (
    pre_df
    .groupby(["survey", "high_exposure"])
    .apply(lambda x: weighted_mean(x, "clean_fuel", "weight") * 100)
    .reset_index(name="clean_fuel_pct")
)

trend_table["group"] = trend_table["high_exposure"].map({
    1: "Treatment: High-exposure states",
    0: "Control: Low-exposure states"
})

trend_table["survey_label"] = trend_table["survey"].map({
    2: "NFHS-2",
    3: "NFHS-3",
    4: "NFHS-4"
})

trend_table = trend_table.sort_values(["high_exposure", "survey"])

display(trend_table)

trend_table.to_csv("outputs/tables/parallel_trends_table.csv", index=False)
```

```
/tmp/ipykernel_6719/3545148973.py:8: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior
  .apply(lambda x: weighted_mean(x, "clean_fuel", "weight") * 100)
```

	survey	high_exposure	clean_fuel_pct	group	survey_label
0	2.0	0	24.637823	Control: Low-exposure states	NFHS-2
2	3.0	0	54.516544	Control: Low-exposure states	NFHS-3
4	4.0	0	33.541444	Control: Low-exposure states	NFHS-4
1	2.0	1	11.125229	Treatment: High-exposure states	NFHS-2
3	3.0	1	72.796267	Treatment: High-exposure states	NFHS-3
5	4.0	1	64.199890	Treatment: High-exposure states	NFHS-4

```
# ============================================================
# 14. PARALLEL TRENDS GRAPH
# ============================================================

plt.figure(figsize=(8, 5))

for group_value, group_name in [
    (1, "Treatment: High-exposure states"),
    (0, "Control: Low-exposure states")
]:
    temp = trend_table[trend_table["high_exposure"] == group_value]

    plt.plot(
        temp["survey_label"],
        temp["clean_fuel_pct"],
        marker="o",
        linewidth=2,
        label=group_name
    )

plt.xlabel("Survey round")
plt.ylabel("Clean fuel adoption (%)")
plt.title("Pre-PMUY Clean Fuel Trends by Treatment Status")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("outputs/figures/parallel_trends_nfhs234.png", dpi=300, bbox_inches="tight")
plt.show()
```